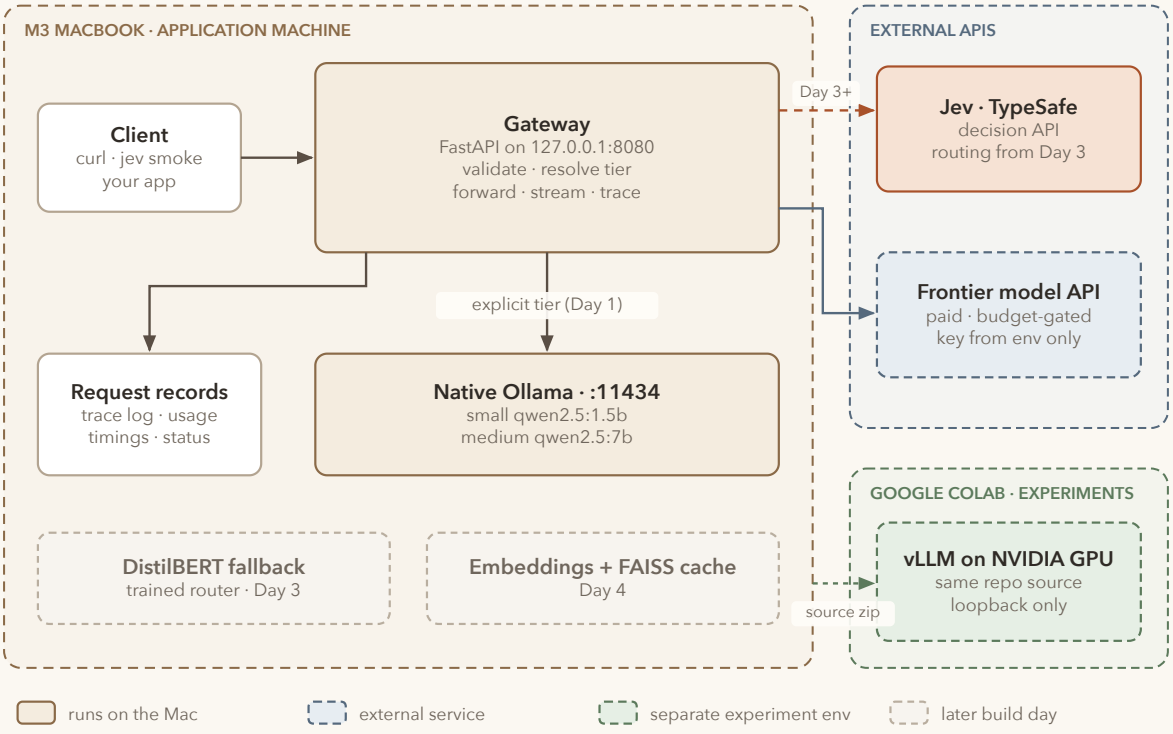
ENGINEERING NOTEBOOK · FOUNDATIONS AND BUILD DAYS 1–2

Jev LLM Gateway

A configurable three-tier LLM gateway with decision-model routing — research, architecture, and the plan and record for the serving foundation and the evaluation baseline.



Project llm_gateway_full_project · package `jev-gateway` 0.1.0
Environments M3 Pro MacBook (application) · Google Colab (NVIDIA experiments) · external APIs
Repository state described Day 1 commits D1-01 to D1-18 (7764899 … 4e6f7e0), 27 Oct 2026 · Day 2 not yet started
Sources accessed 28 October 2026

Contents

How to read this notebook 4
Evidence conventions 4
Short glossary 4

1 Project framing and brainstorming 6
Project brief 6
The problem the gateway solves 6
Why model choice affects quality, latency and cost 6
When routing is useful – and what erases the benefit 7
A useful application versus a benchmark experiment 8
How the project connects to ML systems engineering 8
Questions worth asking before implementation 8
Alternatives considered 9
Constraints and unresolved assumptions 9
Working hypotheses to test 10
Three ways to arrange the runtime 10
Reasons for the proposed architecture 11

2 Background research: how Jev works and how this project uses it 13
Sources and scope 13
What Jev is 13
Generation models and decision models 13
State and typed questions 14
The decision primitives 14
What the returned values actually mean 15
How application code turns a decision into an action 16
Four roles for Jev in this project 16
Tier-choice confidence is not the probability of a correct answer 17
Routing in pseudocode 18
An SDK example checked against current documentation 18
Documented facts and open questions 19

3 What the models and programs are doing 21
The components at a glance 21
Weights, tokens and inference 22
Prefill and decode 22
Context windows and the KV cache 23
Quantization and approximate weight memory 24
Why fitting on disk does not establish runtime feasibility 25
Cold loading, warm execution and model switching 25
Why a larger model is not automatically better on every question 25
Choosing model families: criteria, not conclusions 26

4 Architecture and diagrams 27
A · High-level architecture 27
B · Day 1 request sequence 28
C · How a Jev decision becomes an action 29
D · Day 2 evaluation pipeline 30
Architecture and milestones: consistency check 30

5 **Prerequisites and setup notes** 32
Python packages, imports, virtual environments and lockfiles 32
HTTP requests, JSON, URLs, ports and authentication 32
Why “localhost” differs across a Mac, a container and Colab 33
Async I/O and reusable HTTP clients 33
Timeouts, cancellation and streaming 33
Git: working tree, staging, commits and remotes 34
Unit tests, mocks, contract checks, real integrations and benchmarks 34
Setup sequence on the Mac 35
Setup sequence for Colab 35
Proposed and current repository layout 36

6 **Day 1 — the serving foundation** 38
Overview and recorded state 38

7 **Day 2 — evaluation and baselines** 50
The ideas Day 2 depends on 50
The eighteen Day 2 checkpoints 54

8 **Essential measurement notes** 62
Latency: end to end, first output, and what happens between 62
p50 and p95 63
Request and output-token throughput 63
Accuracy, completion rate and error rate 63
Actual versus modelled cost 64
Cold versus warm runs 64
Mocked versus real evidence 64

Sources 65

How to read this notebook

This notebook explains what the Jev LLM Gateway is, why it is designed the way it is, and how the first two build days turn that design into working software and trustworthy measurements. It is written to be read on its own; the starter guide that accompanies the repository is not required.

The project is larger than two days of work. Six build days are planned, and the later ones add the Jev router, a trained fallback classifier, answer verification, a semantic cache, deployment, load testing and an NVIDIA performance study. Those later features are described here only as far as needed to understand the architecture. The detailed material concentrates on the foundations: background research, the serving gateway (Day 1), and the evaluation and baseline system (Day 2).

Evidence conventions

A notebook like this is only useful if it keeps three kinds of statement apart: what someone else has documented, what has actually been observed in this project, and what is still a plan. Each of these is labelled where it matters.

Label	Meaning in this notebook
DOCUMENTED	Product behaviour stated in official documentation (TypeSafe, Ollama, vLLM, Docker, Colab, dataset cards). Source list on the final pages; all accessed 28 October 2026.
RECORDED	Something the repository itself records as having happened: a commit, a test run, a raw output file under `evidence/`, or an entry in `docs/STATUS.md`. Commit hashes are quoted so each claim can be checked.
PLANNED	Work described in `docs/COMMIT_PLAN.md` that has not been implemented. Everything in Day 2 is planned.
OUR DESIGN	A proposal for how this application will use a product or technique – for example, how a Jev answer is turned into a routing action. These are design choices, not product features.
ILLUSTRATIVE	Numbers invented to explain a calculation or a chart shape. They are never project measurements.

As of the repository state described here, no model has generated an answer through the gateway, no Colab cell has run, and no paid request has been sent. The Day 1 code exists and its offline test suite passes; the live checks are listed as pending. The notebook says so wherever a reader might otherwise assume more.

> **TECHNICAL NOTE · MARGIN CALLOUTS**
>
> Three kinds of callout appear sparingly. **Design decision** records a choice and the alternative it beat. **Working hypothesis** states something the project expects but has not shown; each names how it will be tested. **Technical note** adds a detail needed to read code or numbers correctly.

Short glossary

Tier	One of the three generation choices exposed by the gateway: `small`, `medium`, `frontier`. A tier name is a role, not a guarantee of quality.
Backend	The concrete server behind a tier: a URL, a model identifier, and optionally an API key variable.

Gateway	This project's FastAPI application. It validates requests, resolves a tier to a backend, forwards the call, and records what happened.
Jev	TypeSafe's hosted decision model. It answers typed questions with probabilities; it does not write text.
Router	Code that picks a tier for a request. Jev, DistilBERT, a prompted LLM and fixed or random policies are all routers in the later plan.
Oracle label	For an evaluated prompt, the cheapest tier observed to answer it correctly. Available only after the fact.
SSE	Server-sent events: the `data: …` line format used to stream chat completions over HTTP.
Checkpoint	One planned unit of work with an ID such as `D1-07`, normally one commit.

PART 1

Project framing and brainstorming

Before writing code it is worth being precise about the problem, about what would count as success, and about which parts of the plan are still guesses. This part records that thinking.

Project brief

The Jev LLM Gateway is a single HTTP service that sits in front of several language models and decides, for each request, which one should answer. It exposes one OpenAI-style chat endpoint. Behind it are three generation tiers: a small self-hosted model, a medium self-hosted model, and a paid frontier model reached through a provider's API. On the M3 MacBook, the two self-hosted tiers run in native Ollama; in Google Colab, the same repository code can be pointed at vLLM on an NVIDIA GPU for experiments.

The project is named after Jev, a decision model from TypeSafe AI. Jev does not write answers. It answers narrow, typed questions — "which of these options?", "how far along this scale?", "is this statement true?" — and returns probabilities. The gateway will use those answers to choose a tier, to judge whether a generated answer can be trusted, and to decide whether a cached answer fits a new request. A trained DistilBERT classifier serves as a fallback when Jev is unavailable and as a point of comparison.

Around that core, the full plan adds an evaluation benchmark of 2,000–3,000 prompts, calibration analysis, a verified semantic cache, monitoring, deployment to local Kubernetes, load and failure tests, and an NVIDIA serving experiment. The aim is a system that is useful to run and also a source of honest measurements about when routing pays for itself.

The problem the gateway solves

An application that calls one model for every request makes an implicit bet: that the model is the right trade-off between quality, speed and cost for *every* request it will ever see. That bet is rarely right. "What is 12 × 7?" and "Prove that this recursive function terminates" do not need the same model, yet a single-model application pays the same price and waits the same way for both.

A gateway makes the choice explicit and changeable. Clients talk to one stable endpoint; the gateway owns the list of backends, their credentials, their limits, and the policy for choosing among them. That separation is useful even before any automatic routing exists — it is what Day 1 builds — because it gives one place to enforce timeouts and spending caps and one place to record what every request cost and how long it took.

Why model choice affects quality, latency and cost

Quality. Larger models usually answer more questions correctly, but "usually" hides a lot. Benchmarks show large average gaps on multi-step maths and code, and much smaller gaps on short factual or classification questions. On any given prompt a smaller model can be right where a larger one is wrong. Which prompts are which is an empirical question for Day 2.

Latency. A generating model produces output one token at a time, and each token requires reading the model's weights from memory. More parameters means more bytes to read per token, so on the same hardware a 7-billion-parameter model normally produces tokens more slowly than a 1.5-billion-parameter one. A frontier API adds network time and the provider's queueing, but runs on much larger hardware. On a memory-limited laptop there is a further cost: switching between local models can mean loading several gigabytes from disk.

Cost. Frontier APIs charge per input and output token. Self-hosted models have no per-token bill, but they consume hardware that has a cost whether or not it is busy — electricity and wear on a Mac, or hourly rental for a cloud GPU. The two kinds of cost behave differently and must not be added together as if they were the same unit (Part 7 returns to this).

When routing is useful – and what erases the benefit

Routing is useful when three things are true at once: a meaningful share of requests can be answered correctly by a cheaper or faster tier; something can recognise those requests *before* paying for the expensive answer; and the act of recognising them is cheap relative to what it saves.

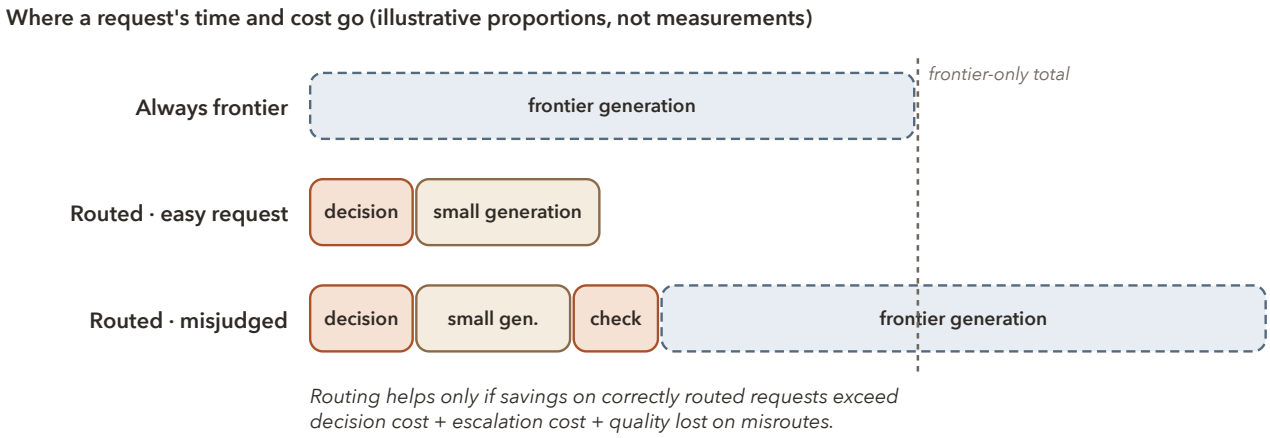


Figure 1. Where time and cost go for a frontier-only request, a correctly routed easy request, and a misjudged request that is escalated after a check. Proportions are illustrative. The misjudged case costs more than not routing at all.

Several overheads can erase the benefit:

- **Decision latency.** A routing call that adds hundreds of milliseconds to every request may cost more user-visible time than it saves on the subset that goes to a faster tier.
- **Misroutes.** Sending a hard request to the small tier either returns a wrong answer (a quality loss) or, with verification, triggers a second generation on a bigger tier (paying twice).
- **Verification calls.** Checking answers improves quality but adds a call per request and turns some single generations into cascades.
- **Model switching.** On a machine that cannot hold both local models in memory, routing between them can force repeated loading — a cost that a benchmark run sequentially per tier would never see.
- **Idle capacity.** If the GPU hosting a self-hosted tier is paid for by the hour, sending it fewer requests does not reduce the bill.

The practical consequence is that routing can only be judged on full-path numbers: every decision, verification and rejected attempt has to be included in the cost and latency of the request it served.

A useful application versus a benchmark experiment

The project has two goals that pull in slightly different directions, and confusing them is a common source of bad conclusions.

As an **application**, the gateway must be dependable: it starts reliably on the Mac, fails clearly when a backend is missing, never leaks a key, never spends beyond a cap, and gives clients a stable API. Its success is measured by whether it behaves correctly under real conditions, including partial failure.

As a **benchmark experiment**, the project must produce numbers that mean something: accuracy per tier with stated sample counts, routing comparisons on a held-out test set, latency under a defined workload, and costs that include everything. Its success is measured by whether another person could reproduce the numbers and would draw the same conclusion.

The two share code but not evidence. A quality evaluation that runs each tier sequentially over thousands of prompts tells us which tier answered which prompt correctly; it does not tell us how fast a live routed request would be, because it never pays for switching, contention or decision calls on the request path. Conversely, a smooth demo of the application says nothing about accuracy. The repository keeps engineering evidence (`docs/STATUS.md`, `docs/EXPERIMENTS.md`, `evidence/`) separate from the code for exactly this reason.

How the project connects to ML systems engineering

The long-term goal behind the project is ML systems and AI infrastructure work — inference optimisation, GPU performance, and making models run efficiently on real hardware. The gateway is a deliberately practical way into that field. It forces contact with the questions that matter in production inference: what happens to memory when a model loads, why the first token is slower than the rest, how batching and concurrency trade latency for throughput, how to measure tail latency honestly, and how to tell a real speed-up from noise.

It also separates two layers that are easy to blur: the *serving runtime* (Ollama, vLLM), which owns memory, batching and kernels; and the *application layer* (this gateway), which owns policy, routing, budgets and records. Day 6's NVIDIA experiment works at the runtime layer — a vLLM baseline, a profile, one evidence-motivated change, and a repeated comparison — while everything before it builds the measurement discipline that makes such an experiment believable.

Questions worth asking before implementation

These are the questions that shaped the plan. Some are answered by the design; several can only be answered by measurement.

- What exactly is the client allowed to send, and what does the gateway refuse? (Day 1 answers: text-only chat; explicit tiers; no images, tools or `auto` yet.)
- Can both local models stay loaded on this Mac at once, and what does switching cost? Unknown until measured with Ollama on the real machine.
- What is a "correct" answer for each task family, and how often does the scorer itself make mistakes?
- What event should a routing probability refer to — "this tier will be correct", or "this tier is the cheapest correct one"? The answer determines how calibration can be measured.
- How much does a Jev call add to request latency from this network, at p50 and p95?

- What is the cheapest possible routing on the measured data (the hindsight oracle), and how close can any real router get?
- What does a wrong answer cost compared with a slow one? This sets the threshold for escalation and cannot come from the data alone.
- Which results belong to the Mac, which to Colab, and how is that labelled so they are never mixed?

Alternatives considered

Decision	Alternative	Why the proposed option was preferred
Local runtime on the Mac	vLLM or llama.cpp server directly	vLLM's GPU path targets Linux with NVIDIA and other accelerators, not the Mac's GPU. Native Ollama is supported on Apple Silicon, manages model downloads and residency, and exposes an OpenAI-compatible endpoint.
API shape	A custom request format	An OpenAI-style `/v1/chat/completions` lets existing clients and tools work, and both Ollama and vLLM speak it, so one transport serves every tier.
Tier selection on Day 1	Start with automatic routing	Routing cannot be evaluated without a working three-tier path and labelled outcomes. Explicit tiers first give a stable base and a clear test target.
Routing signal	Only a prompted LLM router or only a trained classifier	The project compares Jev with a trained DistilBERT, a prompted LLM, a self-hosted decision model and fixed/random baselines on the same split – the comparison is the point.
Where GPU work happens	Rent a permanent GPU server	Colab provides NVIDIA access for bounded experiments without standing infrastructure. Its limits (session lifetime, quota) are acceptable for experiments, not for serving the app.
Budget control	Rely only on the provider dashboard	A process-local admission check stops runaway loops immediately and makes the cap visible in code; provider limits remain the backstop.

Constraints and unresolved assumptions

Hardware. The Mac preflight recorded an Apple M3 Pro with 11 CPU cores, 18 GB of unified memory, macOS 26.5.2, and only 18 GiB free on a 460 GiB data volume RECORDED. That memory is shared by macOS, every open application, the CPU and the GPU. The notebook does not assume that both local models, a classifier, an embedding model, Docker and Kubernetes can be resident together; that has to be measured. Disk space is a real constraint for model downloads.

Colab. Google states that Colab resources are not guaranteed, that idle virtual machines are deleted, and that free notebooks run for at most 12 hours depending on availability DOCUMENTED. GPU type and memory are unknown until a session is allocated.

External access. Jev requires a TypeSafe API key; the frontier tier requires a provider key, a model name, current prices and positive caps. None are configured in the repository yet.

Unresolved assumptions — stated so that they can be checked rather than quietly relied on:

- The chosen Qwen2.5 tags will run acceptably in Ollama on this Mac.
- The three task families will show enough per-prompt disagreement between tiers for routing to matter.
- Jev's probabilities, phrased around this project's events, will carry useful signal about tier success.
- The network latency to Jev and to the frontier provider will be stable enough to measure meaningfully from a home connection.

Working hypotheses to test

WORKING HYPOTHESIS H1 · TIER DISAGREEMENT

On the Day 2 manifest, a substantial share of prompts will be answered correctly by the small tier, and the share will differ sharply between maths, multiple-choice and code. *Test:* per-task accuracy per tier on train and validation splits, with sample counts.

WORKING HYPOTHESIS H2 · ROUTING HEADROOM

The hindsight oracle will cost much less than always-frontier at similar accuracy, leaving room for a real router to save money. *Test:* oracle versus fixed-tier cost and accuracy on validation (Day 2), then each router against that bound (Day 3).

WORKING HYPOTHESIS H3 · DECISION OVERHEAD

A Jev decision adds less latency than the generation time it saves on requests routed to the small tier. *Test:* decision-only p50/p95 and full-path latency on the Mac, with warm-up and model-load state recorded (Day 3).

WORKING HYPOTHESIS H4 · MEMORY RESIDENCY

Both Qwen2.5 tags can be served from Ollama on this Mac, but switching between them under memory pressure will add load time that a sequential evaluation hides. *Test:* `ollama ps`, Activity Monitor memory pressure, and cold/warm/switch timings (D1-08).

Three ways to arrange the runtime

The biggest early decision was where the models run relative to the application. Three arrangements were compared.

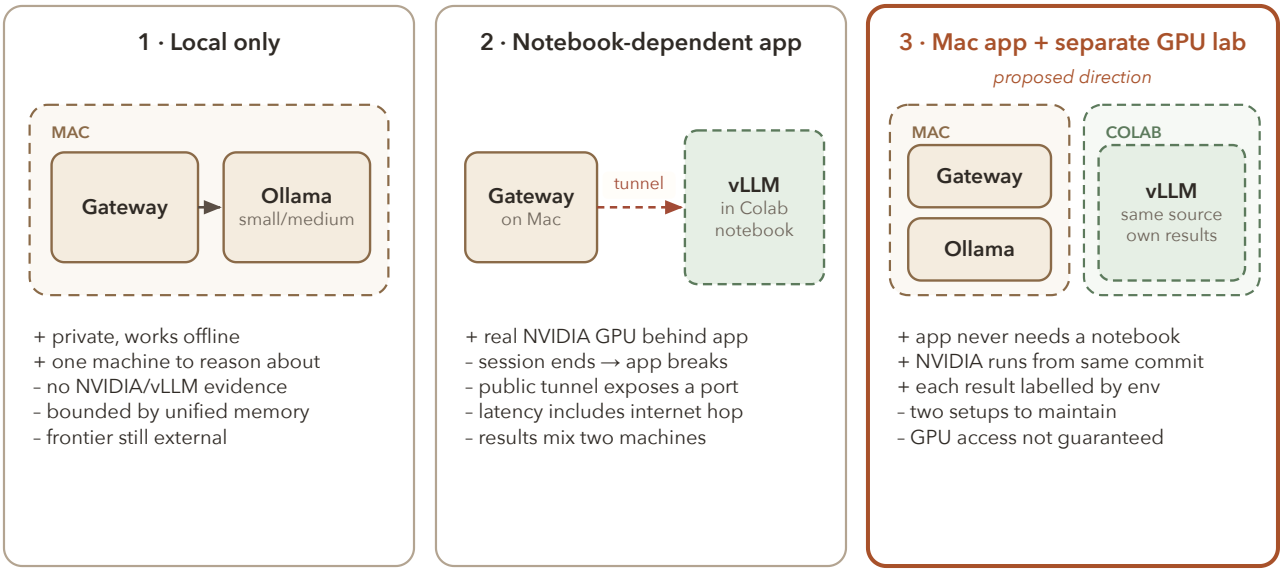


Figure 2. Three runtime arrangements. Option 3 is the proposed direction: the Mac application never depends on a notebook, and Colab runs the same committed source as a separate experiment environment.

Local only. Everything except the frontier and Jev APIs runs on the Mac. It is private, simple, and works without a notebook, but it produces no NVIDIA or vLLM evidence, and it is bounded by 18 GB of shared memory.

Notebook-dependent application. The Mac gateway forwards to a model server running inside a Colab notebook, exposed through a public tunnel. This puts a real GPU behind the app, but the app stops working whenever the session ends; the tunnel exposes a model port to the internet; latency now includes an internet round trip; and results from two machines get mixed into one number.

Mac application plus a separate GPU environment. The Mac runs the application with Ollama. Colab runs the same repository at the same commit, with its own vLLM server on loopback and its own clients, and exports its results. The application never depends on the notebook, and every number is labelled with the environment that produced it. The cost is two setups to maintain and the fact that GPU access is not guaranteed.

> **DESIGN DECISION · PROPOSED RUNTIME**
>
> Use option 3. The gateway's configuration takes a URL per tier, so the same code serves the Mac profile (`127.0.0.1:11434`, Ollama) and the Colab profile (`127.0.0.1:8000`, vLLM inside the notebook). Colab servers stay notebook-local and are never tunnelled. This is a direction, not a conclusion: whether the Mac can host both tiers well is hypothesis H4.

Reasons for the proposed architecture

- **One contract, many backends.** Every tier is "an OpenAI-compatible base URL plus a model name", so adding vLLM, a different provider, or a new local model is configuration rather than code.
- **Policy in the application, not the model server.** Budgets, deadlines, routing and records live in the gateway, where they can be tested offline.
- **Decisions as data.** Jev's answers are numbers that application code interprets, so routing policy stays inspectable and versionable.

- **Evidence by construction.** Each request gets an ID and a trace record; each evaluation row is immutable and fingerprinted. Reports are regenerated from records, never typed in.
- **Install without CUDA.** The core package depends on FastAPI, httpx, Uvicorn and Pydantic only. Training, embeddings and GPU serving are optional extras or notebook-only requirements.

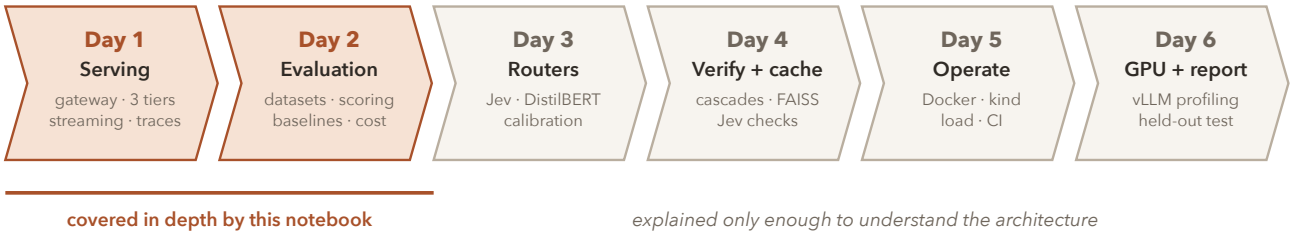


Figure 3. The six planned build days. This notebook covers Days 1 and 2 in detail; later days appear only where they explain the architecture.

PART 2

Background research: how Jev works and how this project uses it

Jev is the component the project is named after, and the one most likely to be misunderstood. This part separates what TypeSafe's official documentation says from the ways this project proposes to use it.

Sources and scope

Everything marked DOCUMENTED in this part comes from TypeSafe's official documentation at `docs.typesafe.ai`, accessed on **28 October 2026**: the Introduction, Quick start, System One, State, Choice, Score, Noul, Confidence, Models, AI primer, Request rules and errors, and Python SDK pages (full list in Sources). Earlier project notes drew on press coverage and used names such as "yes/no" and "scale" questions and a 32K context window; the current official pages name the primitives **Noul** and **Score** and give a 64k per-request and 32k per-question context limit. Where the two disagree, this notebook follows the official documentation and does not repeat unverified press figures for speed or funding.

What Jev is

TypeSafe describes Jev as its flagship model and the first "System One" model: a model built to make fast, structured decisions that software can consume directly DOCUMENTED. The name borrows Daniel Kahneman's distinction between fast, intuitive "System 1" judgement and slow, deliberate "System 2" reasoning. You send Jev a **state** — the material to judge — and a set of **typed questions** about it. It returns one typed answer per question, with probabilities, and no generated prose.

Three documented properties matter most for this project:

- **Constrained answers.** You define the answer space (the options of a Choice, the levels of a Score, yes/no for a Noul). Jev's answer always lies inside that space, so there is no free text to parse.
- **Parallel, isolated questions.** All questions in a request are evaluated in parallel and independently against the same state. TypeSafe states that adding questions barely changes response time, though each adds input tokens.
- **Calibration as a training goal.** TypeSafe says System One models are trained with "reinforcement learning for calibrated decisions" (RLCD) so that probabilities reflect uncertainty, and that calibration "is measured across groups of predictions; it does not guarantee that an individual answer is correct."

Generation models and decision models

A **generation model** — Qwen2.5, a frontier model — takes text and produces new text one token at a time. Its output is open-ended: an explanation, a program, a number embedded in a sentence. To use it for a decision, the application has to coax it into a format and then parse what it wrote, which can fail.

A **decision model** such as Jev takes text and returns a value from a set the caller defined, plus a probability distribution over that set. It cannot answer the user's question; it can only judge something *about* the question or an answer. In this project the two kinds of model have strictly different jobs: Qwen and the frontier model produce answers; Jev helps decide which of them to use and whether to trust what they produced.

> **TECHNICAL NOTE · WHAT JEV NEVER DOES**
> Jev does not call other models, run code, or browse. "Jev routes a request" is shorthand for "the gateway asks Jev a question, then the gateway's own code calls a model". Every action is taken by application code.

State and typed questions

State is the content being judged. The documentation allows a plain string, a JSON object, or an array of text values; it recommends an object for most requests so that each part has a descriptive name, and it describes state as "the material you would present to a panel of experts before asking them to make a judgment." Jev currently accepts text only; English is its primary training language DOCUMENTED.

A **typed question** is a small object with a `type` (`choice`, `score` or `noul`), `instructions` (the question itself), and usually `criteria` (the options, levels, or a description of what "true" and "false" mean). The caller chooses an ID for each question; the answer comes back under that ID, and the model never sees the ID itself. TypeSafe advises keeping each question *atomic* — one judgement — and combining several answers in code rather than asking one compound question.

One System One request: a state, several typed questions, one typed answer per question

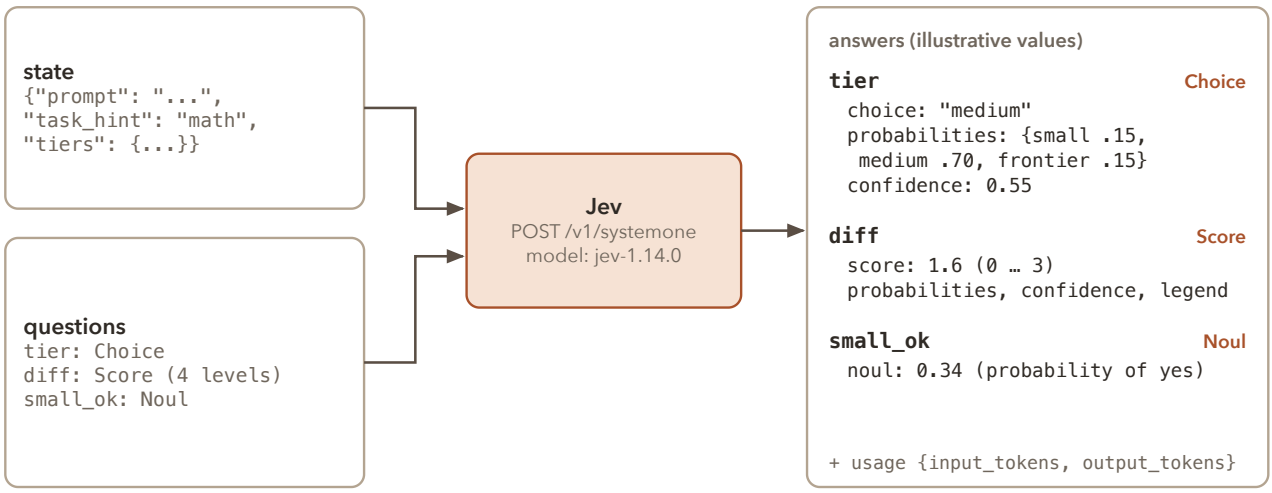


Figure 4. Anatomy of one request to `POST /v1/systemone`. Field names follow the official documentation; the state content, question IDs and answer values are illustrative and were not returned by a real call.

The decision primitives

The current introduction lists three primitives, verified on the pages below. A fourth, **Box**, locates objects in images; it is labelled "dreamer-ready" (not production-ready), requires an opt-in header, and is irrelevant to a text gateway, so it is not used here.

Primitive	Asks	Request criteria	Returned fields
Choice	Which one of these options?	Map of option name → description; up to 255 options	`choice` (option with highest probability), `probabilities` (sum to 1 across options), `confidence` (0–1)
Score	Where on this ordered scale?	Ordered list of 2–10 level descriptions, low to high	`score` (probability-weighted average of level numbers, can fall between levels), `probabilities`, `confidence`, `legend`
Noul	Is this statement true?	Optional descriptions of what true and false mean	`noul`: a single probability that the answer is yes, 0 to 1. No separate confidence field.

Every response also carries the versioned `model` that answered (for example `jev-1.14.0` when the request used the alias `jev-latest`) and `usage` with input and output token counts. TypeSafe notes that aliases move when a new release ships, so a project that tunes thresholds should pin a versioned model ID and log the `model` field of every response DOCUMENTED.

What the returned values actually mean

Probabilities are Jev's distribution over the answer space you defined. For a Choice they are spread over options; for a Score, over levels.

Confidence is *not* a probability of being right. It is a statistic computed from the shape of the distribution: 1 when all probability sits on one outcome, 0 when it is spread evenly. For a Choice with *n* options, the documented formula is `confidence = (p_max - 1/n) / (1 - 1/n)`. For a Score, the formula also accounts for how far probability lies from the most likely level, so indecision between neighbouring levels lowers confidence less than indecision between opposite ends.

```
Illustrative Choice over three tiers:  probabilities = small 0.15, medium 0.70, frontier
0.15
confidence = (0.70 - 1/3) / (1 - 1/3) = 0.3667 / 0.6667 = 0.55

Same top probability, different spread:  small 0.30, medium 0.70, frontier 0.00
confidence = 0.55 as well — the formula only looks at p_max.
```

The **Noul value** is itself the probability of "yes". Near 1 is a strong yes, near 0 a strong no, near 0.5 an even split. TypeSafe suggests starting thresholds of 0.7 for yes and 0.3 for no, sending the middle band elsewhere, and tuning on one's own data rather than to a few test cases. If a confidence-style number is wanted, `|2p - 1|` puts a Noul on the same scale as a Choice.

> **TECHNICAL NOTE · WHAT THE DOCUMENTATION DOES NOT CLAIM**
> The Confidence page makes no calibration claim about `confidence`; it explicitly says right thresholds "depend on your domain and the performance of the model for your use case." The AI primer states calibration as the training objective for probabilities, measured over groups. Neither page says how well calibrated Jev will be for a question like "will this 1.5B model answer this maths problem correctly?" — that is for this project to measure.

How application code turns a decision into an action

The documented pattern is to branch on the answer with risk-scaled thresholds: act automatically at high confidence, proceed carefully at medium, and fall back or escalate at low confidence. TypeSafe's own example gates a destructive action at a higher confidence than a read-only one. In this project the "action" is calling a generation tier, and the risky outcomes are a wrong answer (if the tier is too small) or a needlessly expensive one (if too large). The full flow — context in, typed result out, policy, then the model call — is drawn as architecture diagram C in Part 4 (Figure 14).

The application policy has more inputs than Jev's answer. It must also know whether the chosen tier is enabled (the frontier tier is disabled without a key and caps), whether the budget admits the request, whether the Jev call failed or timed out, and what to do then. Those rules are OUR DESIGN, not product behaviour.

Four roles for Jev in this project

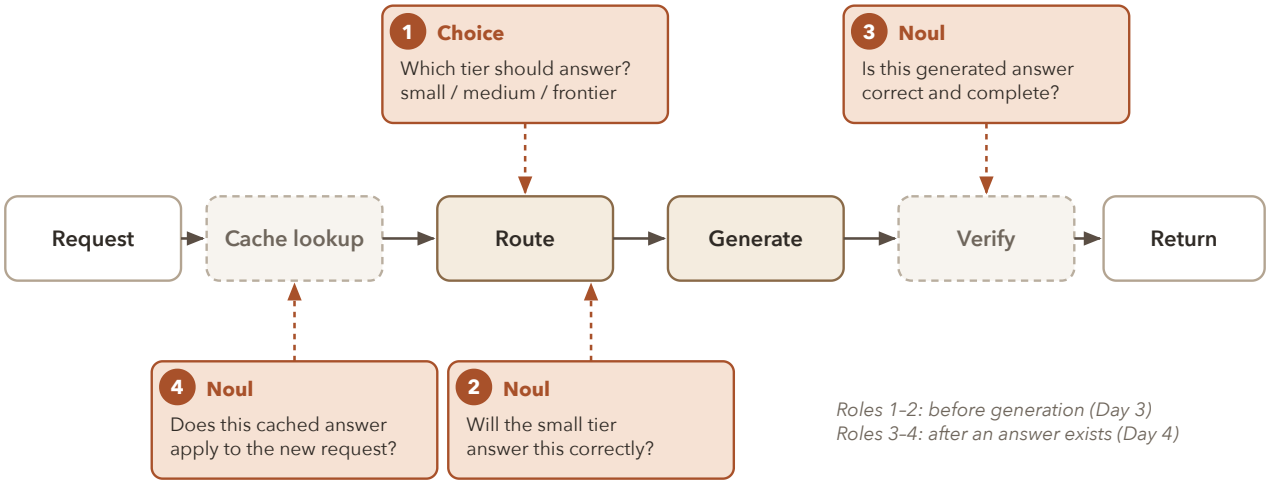


Figure 5. Where each proposed Jev role sits on the request path. Roles 1 and 2 decide before any answer exists; roles 3 and 4 judge an existing answer. All four are application designs to be built and evaluated in Days 3–4.

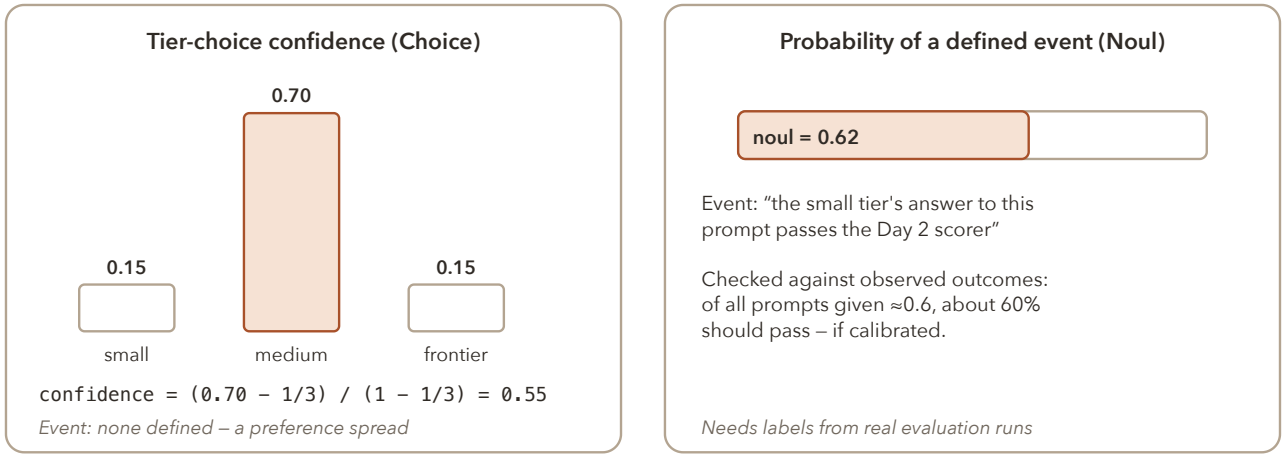
Role	Information supplied	Illustrative question	Result	Gateway's next step · limitation to evaluate
1 · Select a tier before answering	State: the prompt and short descriptions of each tier's strengths	Choice: "Which is the cheapest model likely to answer the `prompt` correctly?" options small / medium / frontier	`choice`, probabilities, confidence	Call the chosen tier if confidence clears a threshold and the tier is available; otherwise fall back. *Limitation:* one fixed operating point; tier descriptions must be tuned on validation; confidence measures spread, not correctness.
2 · Estimate whether a named backend will succeed	State: the prompt and a description of one specific tier	Noul: "Would a small 1.5-billion-parameter instruction model answer the `prompt` correctly?"	`noul` probability	Use small if `noul` $\geq t_1$; else ask about medium; else frontier. Sweeping t gives a cost–quality curve. *Limitation:* asking twice adds latency and tokens; calibration must be measured against observed success labels.

Role	Information supplied	Illustrative question	Result	Gateway's next step · limitation to evaluate
3 · Check an already generated answer	State: the prompt and the generated answer	Noul: "Does the `answer` correctly and completely respond to the `prompt`?"	`noul` probability	Return the answer if above threshold; else escalate to the next tier (a cascade). *Limitation:* false accepts return wrong answers; false rejects pay twice; escalation must not replay output already streamed.
4 · Check that a cached answer applies	State: the new prompt, the cached prompt, the cached answer	Noul: "Does the `cached_answer` correctly answer the `new_prompt`?"	`noul` probability	Serve the cached answer only on a confident yes; else generate. *Limitation:* near-miss pairs ("capital of Australia" vs "largest city of Australia") are where similarity search fails and the check must succeed; false-hit rate needs its own benchmark.

A request can combine roles. Because questions in one request run in parallel, roles 1 and 2 could be asked together in a single call; roles 3 and 4 need the answer or cache candidate first, so they are always separate calls on the request path, and their latency adds to the request.

Tier-choice confidence is not the probability of a correct answer

It is tempting to read "Jev chose medium with confidence 0.9" as "there is a 90% chance the medium tier's answer will be correct". That reading is wrong for two separate reasons. First, `confidence` is a measure of how concentrated the Choice distribution is, not a probability of anything. Second, even the underlying Choice probability refers to the question that was asked — "which is the cheapest model likely to answer correctly?" — which is a different event from "the medium tier will be correct". A prompt every tier answers correctly should, ideally, make Jev choose *small*; a prompt no tier can answer has no right choice at all.



A confident tier choice is not a probability that the chosen tier's answer will be correct. Only the right panel names an event that can be scored.

Figure 6. Left: a Choice over tiers yields a preference distribution and a dispersion-based confidence; no scoreable event is defined. Right: a Noul phrased around a specific, scoreable event can be compared with observed outcomes. Values are illustrative.

Calibration needs a defined event and observed labels. To say a probability is calibrated, one needs (a) an event that is precisely defined and objectively decidable — for example, "the small tier's answer to this prompt, generated with these settings, passes the Day 2 scorer" — and (b) the actual outcome of that event for many prompts. Day 2 produces exactly those labels: for every prompt in the manifest and every tier, whether the answer was correct. With them, Day 3 can group Noul predictions into probability bins and compare each bin's average prediction with the fraction of prompts that actually succeeded.

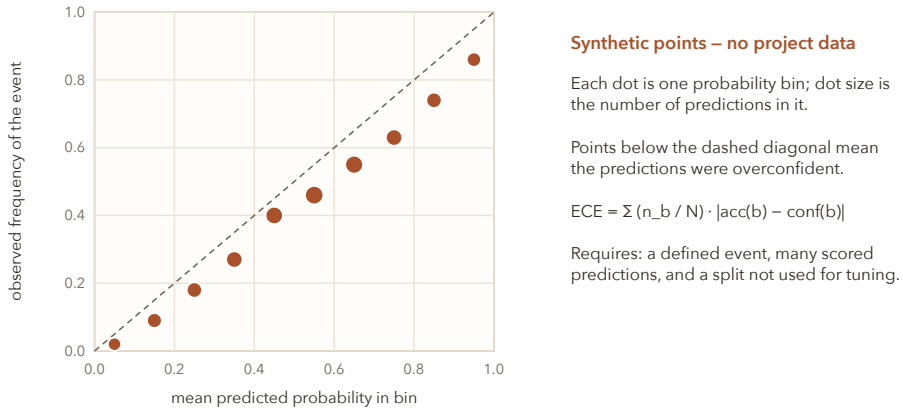


Figure 7. How calibration will be read once labels exist: a reliability diagram plots observed frequency against mean predicted probability for each bin. The points shown are synthetic and say nothing about Jev.

Routing in pseudocode

PSEUDOCODE · THRESHOLD ROUTING WITH NOUL QUESTIONS (ROLE 2) · OUR DESIGN, NOT IMPLEMENTED

```
function route(request, deadline):
    if request.model != "auto": return request.model          # Day 1 behaviour: explicit tier
    state = { prompt: last_user_message(request), tiers: TIER_DESCRIPTIONS_V2 }
    questions = { small_ok:  Noul("Would the small tier answer `prompt` correctly?"),
                  medium_ok: Noul("Would the medium tier answer `prompt` correctly?") }
    try:
        answers = jev.ask(state, questions, timeout = min(1.0 s, deadline.remaining))
    except (Timeout, ApiError):
        return fallback_router(request)                        # DistilBERT, Day 3
    record(question_version = "v2", answers, latency)
    if answers.small_ok  >= T_SMALL  and enabled("small"):  return "small"
    if answers.medium_ok >= T_MEDIUM and enabled("medium"): return "medium"
    if enabled("frontier") and budget.admits(request):      return "frontier"
    return "medium"                                            # never silently upgrade to paid
```

Both questions are asked in one call because they are independent and parallel. `T_SMALL` and `T_MEDIUM` are chosen on the validation split and then frozen before the held-out test.

An SDK example checked against current documentation

The example below uses only names that appear in the official Python SDK documentation: `AsyncTypeSafeClient`, `Choice`, `RetryPolicy(max_retries, backoff_max, timeout)`, `client.system_one(state=…, questions=…, retry=…)`, `response.model`, `response.usage`,

`response.answers[id].choice / .probabilities / .confidence`, and `TypeSafeAPIError` with `status` and `request_id`. The package is `typesafe-sdk` (Python 3.10+), and the client reads `TYPESAFE_API_KEY` from the environment. It has not been run: no TypeSafe key is configured for this project.

PYTHON · VERIFIED AGAINST DOCS.TYPESAFE.AI ON 28 OCT 2026 · NOT EXECUTED

```python
from typesafe_sdk import AsyncTypeSafeClient, Choice, RetryPolicy, TypeSafeAPIError

TIER_QUESTION_V1 = Choice(
    instructions="Which is the cheapest model likely to answer the `prompt` correctly?",
    criteria={
        "small": "1.5B instruction model: short facts and simple arithmetic",
        "medium": "7B instruction model: multi-step reasoning and short programs",
        "frontier": "Large hosted model: hard maths, long or subtle code",
    },
)

async def choose_tier(client: AsyncTypeSafeClient, prompt: str) -> dict:
    try:
        response = await client.system_one(
            state={"prompt": prompt},
            questions={"tier": TIER_QUESTION_V1},
            retry=RetryPolicy(max_retries=1, backoff_max=0.2, timeout=1.0),
        )
    except TypeSafeAPIError as error:
        return {"fallback": True, "status": error.status, "request_id": error.request_id}
    answer = response.answers["tier"]
    return {"jev_model": response.model,                # versioned ID, e.g. jev-1.14.0
            "input_tokens": response.usage.input_tokens,
            "choice": answer.choice,
            "probabilities": dict(answer.probabilities),
            "confidence": answer.confidence}

# usage:  async with AsyncTypeSafeClient(model="jev-1.14.0") as client: ...
```

Two details are worth noting. Pinning `jev-1.14.0` instead of `jev-latest` stops thresholds from silently drifting when the alias moves. And the SDK retries `429` and `529` responses by default; on a request path with a deadline, the retry budget must be bounded, which is why the example sets it explicitly.

Documented facts and open questions

Documented (accessed 28 Oct 2026)	Not documented – must be measured or remains unknown
Endpoint `POST https://api.typesafe.ai/v1/systemone`; Bearer key in `Authorization`.	Latency from this Mac's network to the API, at p50 and p95, under the project's request sizes.
Models Jev 1.14 (`jev-1.14.0`, available from 14 Oct 2026) and Jev 1.13; aliases `jev-latest`, `jev-preview`.	Jev's internal architecture, parameter count and serving hardware.
Price: pay per input token, $0.042 per million; output tokens free.	Total Jev spend for this project – depends on state size and question count per request.
Context: 64k tokens per request, 32k per question (state plus one question); an opt-in long-context header raises both, documented as slower and less accurate.	Accuracy as state grows on this project's prompts (TypeSafe publishes a "jaggedness" page for 1.13 that is not reproduced here).

Documented (accessed 28 Oct 2026)	Not documented — must be measured or remains unknown
Training objective: RLCD for calibrated probabilities; calibration holds over groups, not individual answers. Same weights for every account; not trained on customer requests.	Calibration for this project's events (tier success, answer correctness, cache applicability).
Errors: 400, 401, 413, 415, 422, 429 (rate limit), 529 (overloaded); SDKs retry 429/529 with backoff.	The organisation's actual rate limits (shown only in the TypeSafe console).
Text input only for production use; English is the primary language.	How well Jev judges maths- and code-heavy prompts, given guidance that it handles words better than raw numbers (from earlier project notes, not re-verified).

PART 3

What the models and programs are doing

The project involves six kinds of model and five kinds of program. Keeping clear which is which — weights, the software that runs them, and the application that decides what to ask — prevents most of the confusion that follows.

The components at a glance

Component	Kind	Runs where	Input → output	Role in the project
Small model	Generation model (weights)	Mac via Ollama; Colab via vLLM	chat messages → text	Cheapest, fastest tier. Current candidate: Qwen2.5 1.5B Instruct (`qwen2.5:1.5b`).
Medium model	Generation model	Mac via Ollama when memory permits; Colab via vLLM	chat messages → text	Middle tier for multi-step tasks. Candidate: Qwen2.5 7B Instruct (`qwen2.5:7b`).
Frontier model	Generation model behind a paid API	Provider's servers	chat messages → text + usage	Highest-quality, most expensive tier. Model and provider not yet configured.
Jev	Decision model behind an API	TypeSafe's servers	state + typed questions → typed answers + probabilities	Principal router and verifier (Days 3–4). Never generates the reply.
DistilBERT	Classifier model, fine-tuned by the project	Trained on Colab; served on Mac CPU	prompt text → tier label + probabilities	Fallback router when Jev is unavailable; comparison baseline (Day 3).
Embedding model	Encoder model	Mac	text → fixed-length vector	Finds similar past prompts for the semantic cache (Day 4).
Ollama	Serving software	Mac (native app, port 11434)	HTTP requests → model execution	Downloads, loads, unloads and runs the local models; OpenAI-compatible endpoint.
vLLM	Serving software	Colab NVIDIA GPU (port 8000)	HTTP requests → batched GPU execution	High-throughput serving for NVIDIA experiments (Day 1 notebook, Day 6 study).
Gateway	Application code (this repo)	Mac, `127.0.0.1:8080`	client request → backend call → response + trace	Validates, resolves a tier, forwards, streams, enforces caps, records.
Client	Application code / tool	Mac	user intent → HTTP request	`curl` , `jev smoke` , later the evaluation runner and load tests.
Colab notebook	Experiment harness	Colab VM	uploaded source → server, requests, exported JSON	Runs the same committed source against vLLM; exports results; never a backend for the Mac.

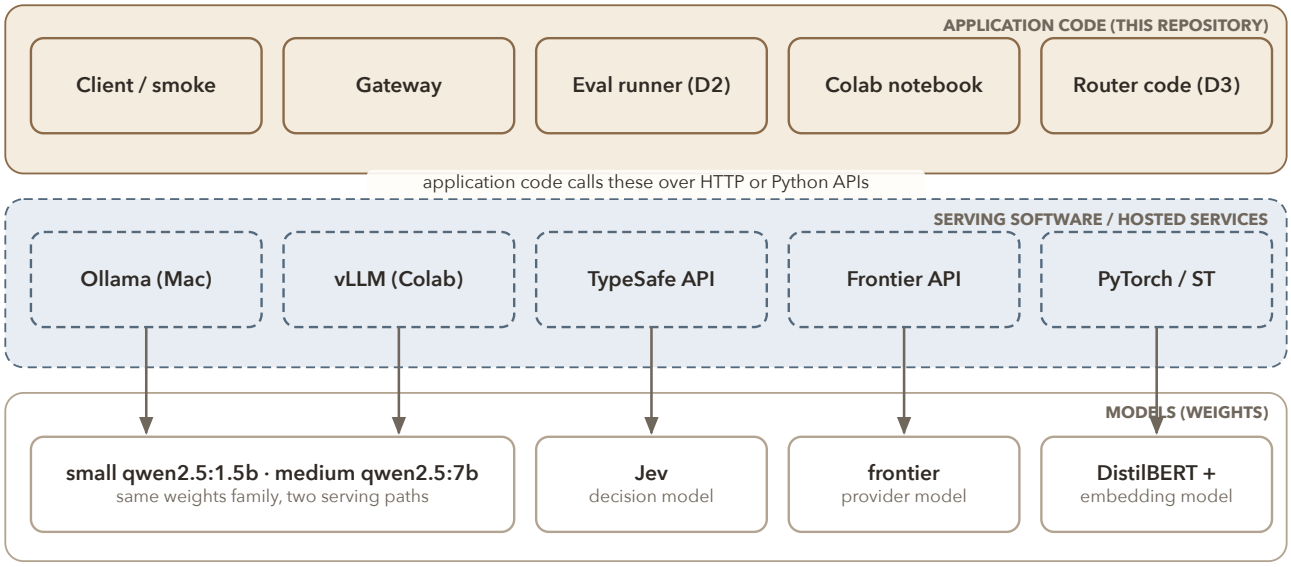


Figure 8. Three layers. Application code decides what to ask and records what happened; serving software and hosted services execute models; models are the trained weights. The same Qwen2.5 family is reached through two serving paths, Ollama on the Mac and vLLM on Colab.

Weights, tokens and inference

A trained model is a large set of numbers called **weights** (or parameters). Qwen2.5 7B Instruct has about 7.6 billion of them; the 1.5B model about 1.5 billion. The weights are fixed after training; using the model does not change them.

Models do not read characters directly. A **tokenizer** splits text into **tokens** — whole words, word pieces, punctuation — and maps each to an integer ID. "The gateway streams tokens." might become six or seven tokens. Prices, context limits and speeds are all counted in tokens, and different models' tokenizers split the same text differently, which is why the gateway's budget check estimates conservatively rather than counting exactly.

Inference means running the model forward on an input to get an output. For a generation model, the output of one forward pass is a probability distribution over the next token. The server picks a token (always the most likely one at temperature 0, or a random draw weighted by probability at higher temperatures), appends it, and repeats until it produces an end token or reaches `max_tokens`.

Prefill and decode

A generation request happens in two phases with very different performance characteristics.

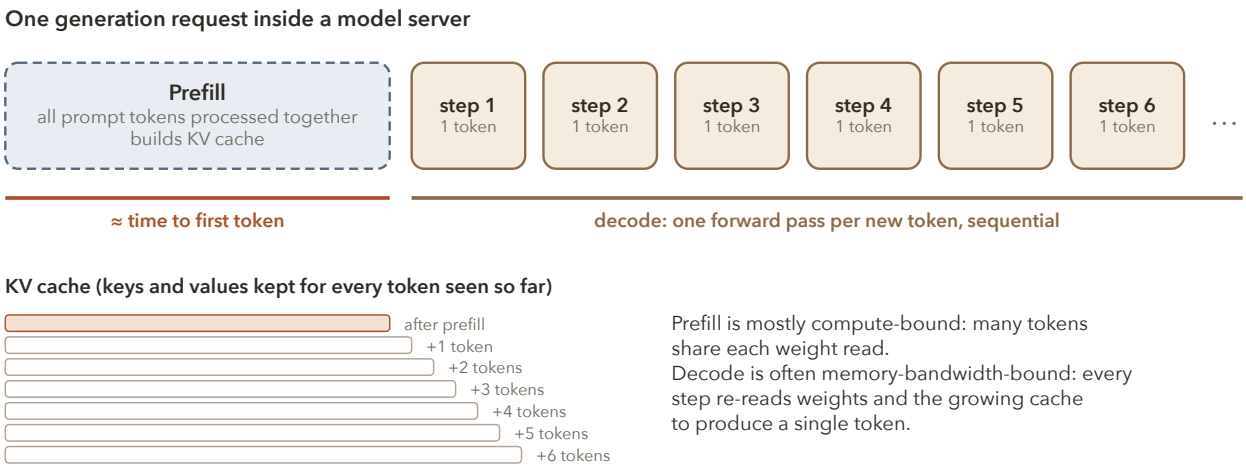


Figure 9. Prefill processes the whole prompt at once and builds the KV cache; decode then produces one token per step, each step reading the weights and the growing cache.

Prefill processes every prompt token in one batched pass. Because many tokens share each read of the weights, prefill keeps the processor busy with arithmetic; its time grows with prompt length. It ends with the first output token, so prefill dominates the *time to first token*.

Decode generates the remaining tokens one at a time, because each new token depends on the previous one. Every decode step must stream essentially all the weights through the processor to produce a single token, so decode speed on one request is usually limited by memory bandwidth rather than arithmetic. This is why serving systems batch decode steps from many requests together: one read of the weights then produces a token for each request in the batch. vLLM's design centres on doing that efficiently; it is also where the Day 6 GPU experiment will look.

```
Illustrative arithmetic, not a measurement
prompt = 300 tokens,  output = 120 tokens
if prefill takes 0.15 s and decode runs at 40 tokens/s:
  time to first token ≈ queue + 0.15 s
  remaining 119 tokens ≈ 119 / 40 = 2.98 s
  end-to-end ≈ 0.15 + 2.98 ≈ 3.1 s   → output length, not prompt length, dominates
```

Context windows and the KV cache

The **context window** is the maximum number of tokens — prompt plus generated output — a model can attend to in one request. Qwen2.5's published configuration allows 32,768 positions, but a serving runtime may allocate less: Ollama documents a default context of 4,096 tokens, adjustable with `OLLAMA_CONTEXT_LENGTH` or a Modelfile `DOCUMENTED`. Text beyond the configured window is truncated or rejected, so a "fits the model" claim must name the runtime setting.

During attention, each layer computes a *key* and a *value* vector for every token. Recomputing them for all earlier tokens at every decode step would be wasteful, so the server keeps them in the **KV cache**. The cache grows by a fixed amount per token, per concurrent request.

```
KV cache per token = 2 (K and V) × layers × KV heads × head size × bytes per value
Qwen2.5-7B-Instruct config: 28 layers, 4 KV heads, head size 3584/28 = 128, bf16 = 2 bytes
  2 × 28 × 4 × 128 × 2 B = 57,344 B ≈ 56 KiB per token
  × 4,096 tokens  ≈ 224 MiB per full-context request
  × 32,768 tokens ≈ 1.75 GiB per full-context request
Qwen2.5-1.5B-Instruct: 28 layers, 2 KV heads, head size 128 → 28 KiB per token, ≈ 112 MiB
at 4,096
(Configs from the Hugging Face model repositories. Runtimes may store the cache at lower
precision.)
```

Ollama notes that memory needed scales with `OLLAMA_NUM_PARALLEL × OLLAMA_CONTEXT_LENGTH`, which is this same arithmetic: allowing four parallel 4,096-token requests on the 7B model reserves roughly four times 224 MiB for the cache alone.

Quantization and approximate weight memory

Quantization stores weights with fewer bits — 8 or 4 instead of 16 — trading a small amount of accuracy for less memory and less data to read per token. The first-order memory estimate is simple:

```
weight bytes ≈ parameters × bits per weight ÷ 8
Qwen2.5-7B: 7,615,616,512 params × 16 bits ÷ 8 ≈ 15.2 GB     × 4 bits ÷ 8 ≈ 3.8 GB
Qwen2.5-1.5B: 1,543,714,304 params × 16 bits ÷ 8 ≈ 3.1 GB    × 4 bits ÷ 8 ≈ 0.77 GB
Ollama's listed downloads: qwen2.5:7b 4.7 GB, qwen2.5:1.5b 986 MB — larger than pure 4-bit,
because practical 4-bit formats keep some tensors at higher precision and add scales and
metadata.
```

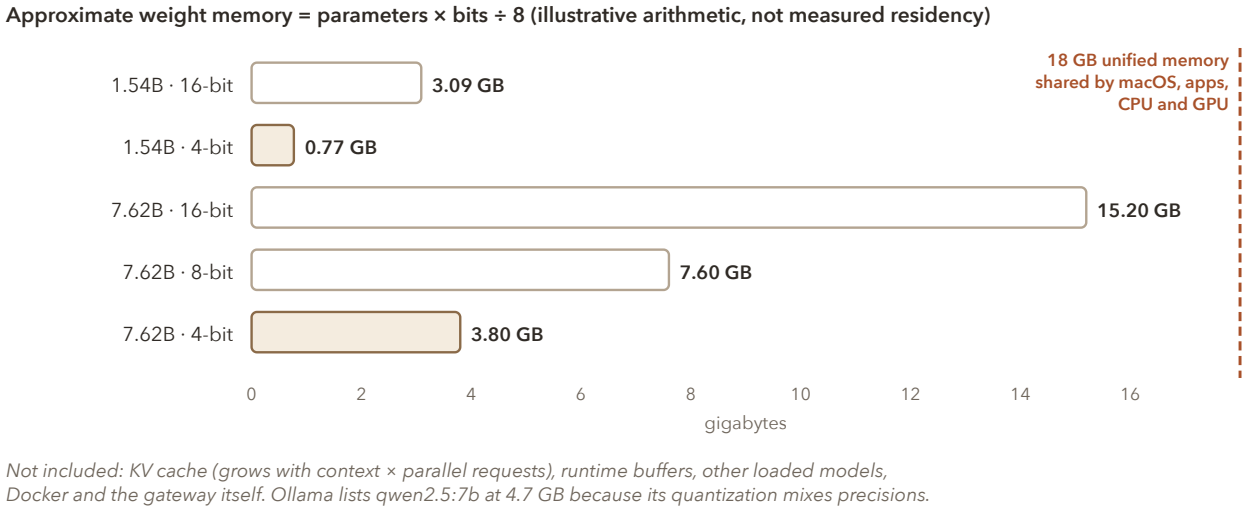


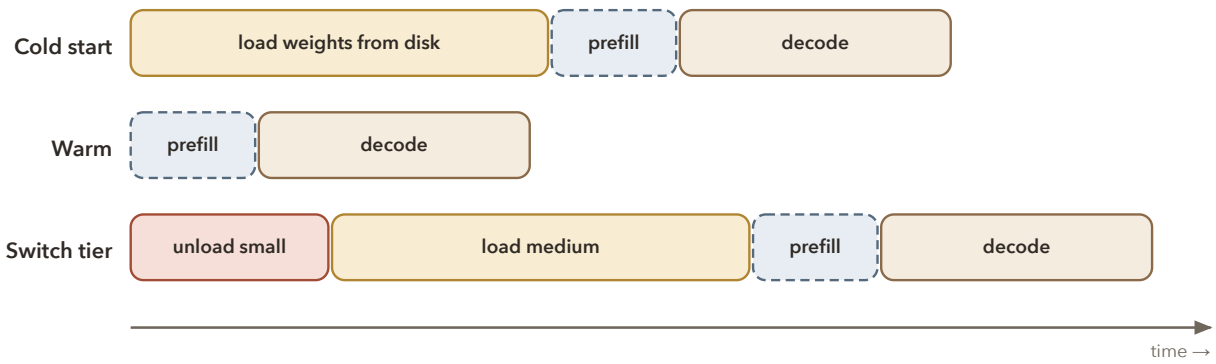
Figure 10. First-order weight memory for the two candidate models at three precisions, against the Mac's 18 GB of unified memory. The bars are arithmetic, not measured residency.

The Colab notebook serves `Qwen/Qwen2.5-1.5B-Instruct` from Hugging Face in its native 16-bit form, while the Mac serves Ollama's quantized build. They are the same model family but not the same artefact, so quality and speed results from the two environments are not interchangeable.

Why fitting on disk does not establish runtime feasibility

A 4.7 GB download proves only that the file fits on disk. At runtime the model also needs the KV cache for each active request, scratch buffers for intermediate activations, the runtime's own code and graph, and room for everything else on the machine. On Apple Silicon the CPU and GPU share one pool of unified memory, and macOS reserves part of it; Ollama may also place part of a model on the CPU if the GPU share is insufficient (its `ollama ps` output shows the split). The only trustworthy evidence is an observation: load the model, run a request at the intended context length, and read `ollama ps`, the `size_vram` field of `/api/ps`, and memory pressure in Activity Monitor. The repository records the download sizes and explicitly makes no residency claim yet.

Cold loading, warm execution and model switching



Block widths are illustrative. Ollama keeps a model loaded for 5 minutes by default (keep_alive); when memory is short, new requests queue until idle models unload.

Figure 11. A cold request pays for loading weights before prefill; a warm request does not. When two models cannot both stay resident, switching tiers can mean unloading one and loading the other.

A **cold** request arrives when the model is not in memory: the server must read gigabytes from disk and initialise the runtime before prefill begins. A **warm** request finds the model already loaded. By default Ollama keeps a model loaded for five minutes after its last use (the `keep_alive` setting), may keep up to three models loaded on CPU-based or single-GPU systems if they fit, and queues new requests while it unloads idle models when memory is short DOCUMENTED.

This matters twice. For measurement, the first request after a load is not representative, so benchmarks need a warm-up and must record load state. For routing, a policy that alternates between small and medium on a machine that can hold only one of them will pay load time repeatedly — a cost invisible in an evaluation that runs all small requests and then all medium ones.

Why a larger model is not automatically better on every question

- **Averages hide per-prompt reversals.** A larger model with higher average accuracy can still miss prompts a smaller model gets right; the oracle table in Part 7 is built to expose such rows.
- **Format and length.** A model that explains at length may hit `max_tokens` before stating the final answer, which the scorer will record as truncated rather than correct. The gateway's default output cap is 256 tokens.
- **Different artefacts.** A heavily quantized larger model may lose more accuracy than a lightly quantized smaller one.

- **The question may be easy.** For simple facts and classification, extra capacity adds latency and cost without adding correctness.
- **Speed is part of quality.** An answer that arrives too late for its deadline is a failure, regardless of its content.

Choosing model families: criteria, not conclusions

The original plan sets role targets — a small instruction model of about 1–3B parameters and a quantized medium model of about 7–8B — and leaves the specific choice to the hardware. The current candidates, Qwen2.5 1.5B and 7B Instruct, were selected after the disk and memory preflight. Both are published under the Apache-2.0 licence on their Hugging Face model pages, both are available as Ollama tags, and the same family can be served by vLLM in Colab, which keeps the Mac and NVIDIA paths comparable in kind.

The choice remains conditional. A model should be replaced if it does not run acceptably on the actual Mac, if its runtime support changes, if its licence terms do not fit the intended use, or if newer documentation recommends a different checkpoint. The criteria, in order: (1) measured memory and latency on the target machine; (2) supported by Ollama on Apple Silicon and by vLLM on the allocated GPU; (3) an instruction-tuned chat template; (4) licence compatible with publishing results; (5) the same family available in both environments; (6) a model card that states context length and revision.

PART 4

Architecture and diagrams

Four diagrams carry the architecture: the system as a whole (A), the Day 1 request path (B), how a Jev decision becomes an action (C), and the Day 2 evaluation pipeline (D). Each is followed by the reasoning it summarises.

A · High-level architecture

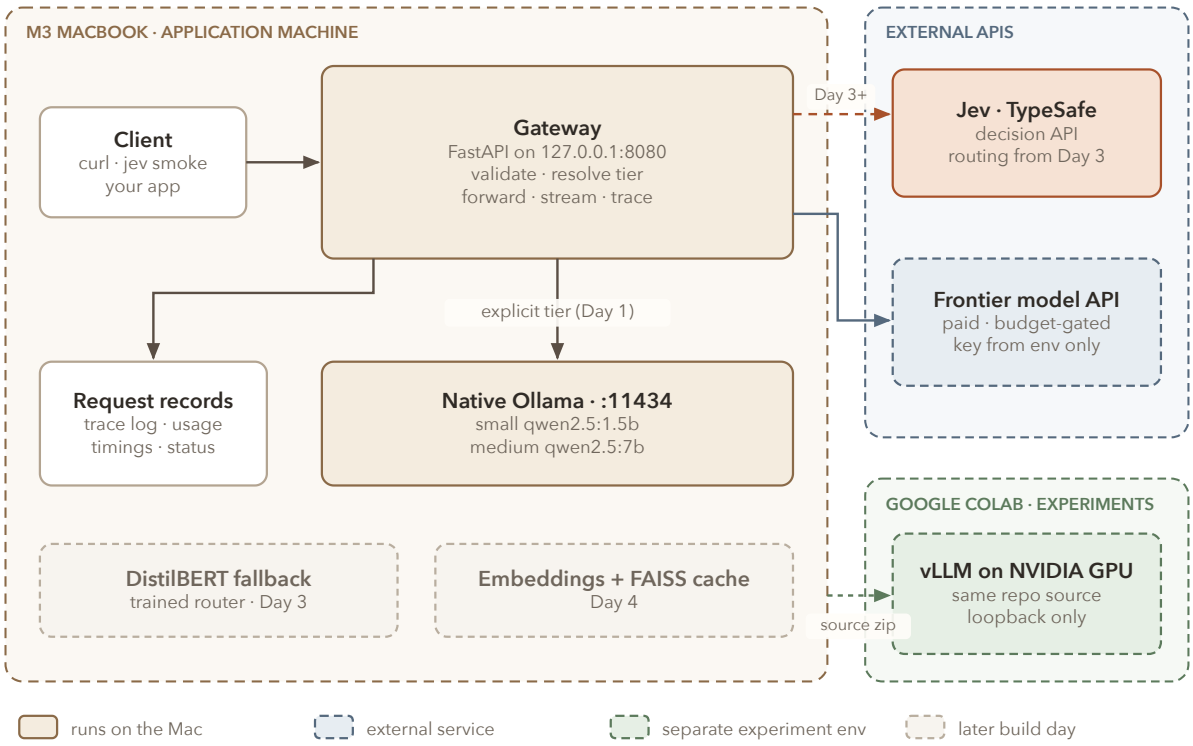


Figure 12. Diagram A. Components on the Mac (solid warm boxes), external services (blue dashed), the separate Colab experiment environment (green dashed), and components added on later build days (grey dashed).

The **client** — `curl`, `jev smoke`, or any OpenAI-compatible tool — sends a chat request to the **gateway** on `127.0.0.1:8080`. The gateway validates the request, resolves the tier to a backend, and forwards it. For the small and medium tiers on the Mac, the backend is **native Ollama** on port 11434, which serves both models from one process and decides which ones stay loaded. For the frontier tier, it is a paid provider API reached over HTTPS with a key read only from the environment. Every request produces a **trace record** with an ID, timings, status and token usage.

Jev appears on the diagram from the start because it shapes the architecture, but the gateway does not call it until Day 3. The trained DistilBERT fallback and the embedding cache are drawn as later components on the Mac. **Colab** is drawn apart on purpose: it receives a source archive and runs its own vLLM server and clients on loopback inside the VM. Nothing on the Mac sends requests to it.

B · Day 1 request sequence

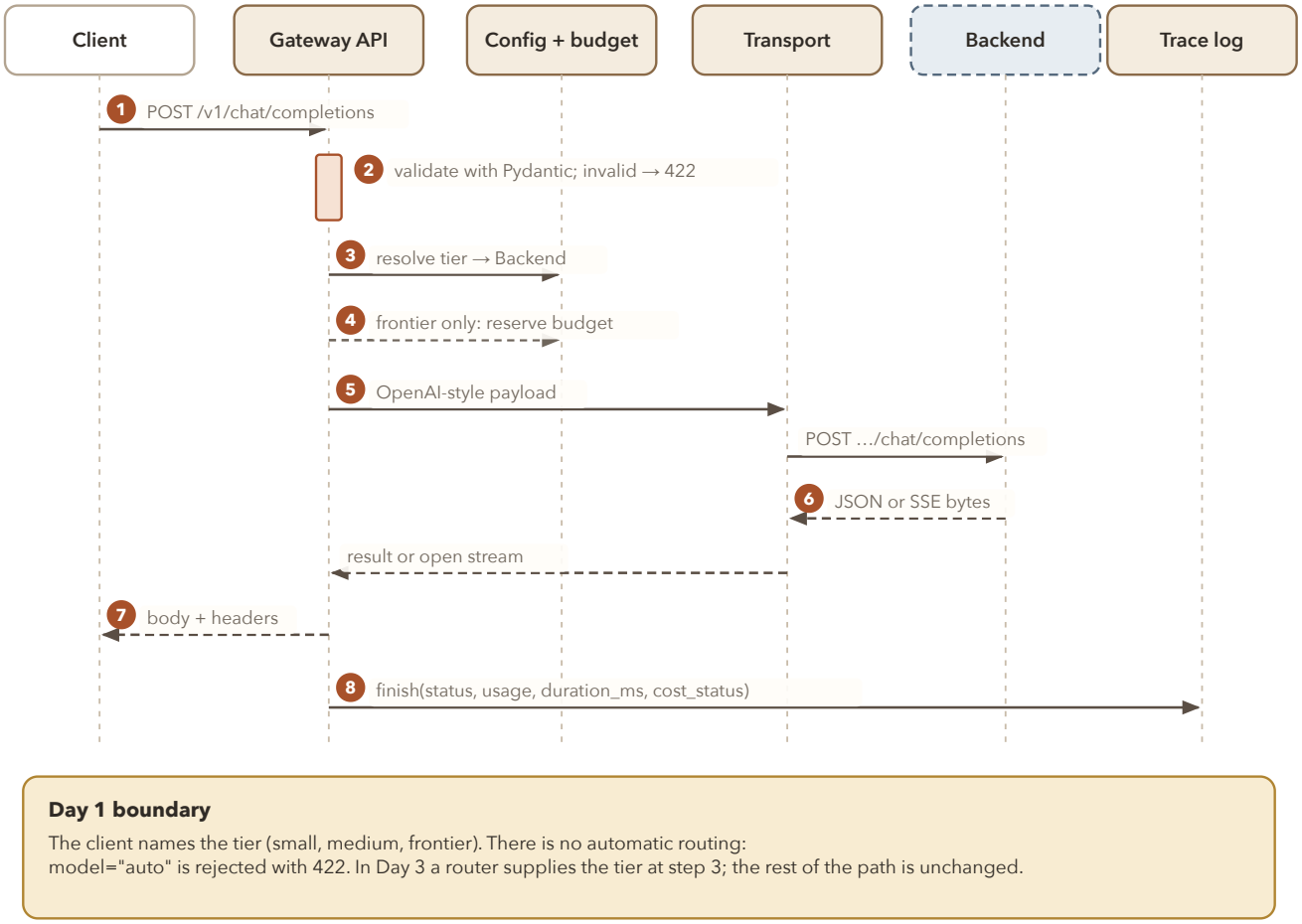


Figure 13. Diagram B. One Day 1 request with an explicit tier. Steps 1–8 correspond to code in `api.py`, `config.py`, `budget.py`, `transport.py` and `tracing.py`. Dashed return arrows carry response data.

1. **Client request.** `POST /v1/chat/completions` with `model` set to `small`, `medium` or `frontier`.
2. **Validation.** Pydantic models reject anything outside the contract — unknown fields, non-text content, empty message lists, `model: "auto"` — with HTTP 422 before any backend is touched.
3. **Backend resolution.** `Settings.backend(tier)` returns the configured URL and model name. For the frontier tier, the gateway first checks that it is enabled (key, model, prices and positive caps) and returns 503 if not.
4. **Budget admission** (frontier only). `FrontierBudget.reserve` estimates tokens and dollars and returns 429 if the process caps would be exceeded.
5. **Forwarding.** The shared `BackendTransport` posts an OpenAI-style payload to `{base_url}/chat/completions`.
6. **Backend response.** Either a complete JSON body or, when streaming, a stream of SSE bytes.
7. **Response to the client.** The backend's JSON is returned unchanged; routing metadata travels in headers: `x-request-id`, `x-jev-tier`, `x-jev-backend-model`.
8. **Outcome record.** `RequestTrace.finish` logs one JSON line with status (`ok`, `timeout`, `backend_error`, `stream_interrupted`), duration, usage and cost status.

DESIGN DECISION · KEEP THE BACKEND BODY INTACT

The gateway does not rewrite the backend's completion JSON or add fields to it. Metadata goes in headers, so any OpenAI-compatible client keeps working, and the original usage numbers survive untouched for later cost accounting. The alternative — a gateway-specific envelope — would have broken client compatibility for little gain.

C · How a Jev decision becomes an action

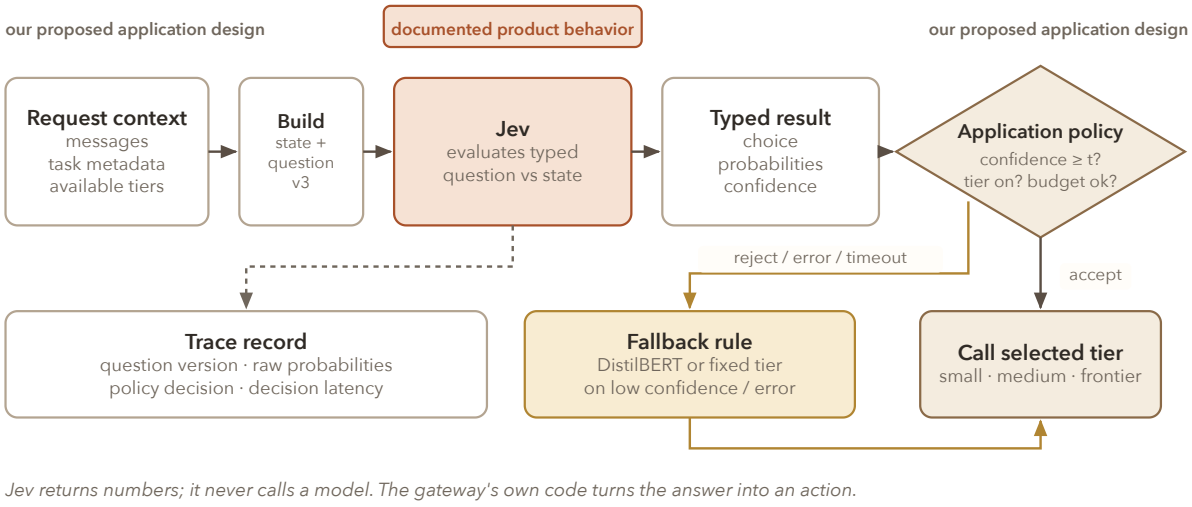
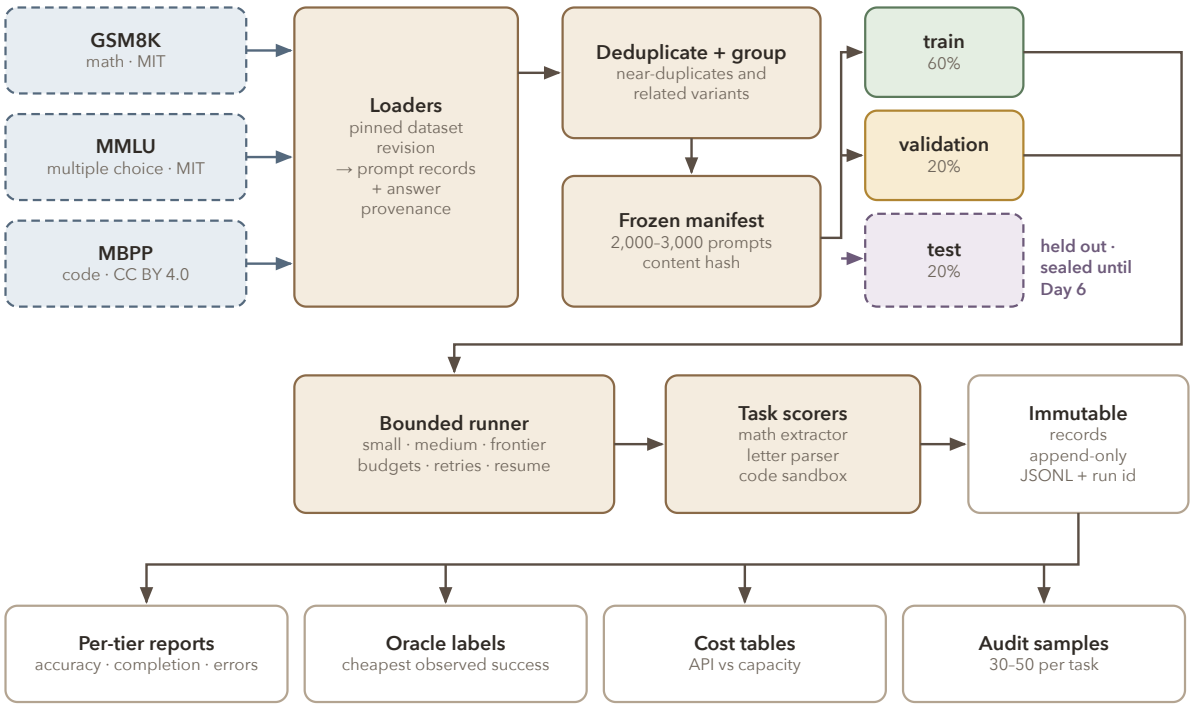


Figure 14. Diagram C. The request context is turned into a state and a versioned question; Jev returns a typed result; application policy interprets it together with availability and budget; then the gateway calls the selected tier or falls back. Only the Jev box is product behaviour.

Each step to the left and right of Jev belongs to this project. The **state builder** decides what Jev sees — the user's last message, perhaps a task hint, and plain-language descriptions of each tier — and stamps the question with a version so that results can be traced to wording. **Policy** combines Jev's answer with facts Jev cannot know: which tiers are enabled, what the remaining budget allows, how long the request has left before its deadline. When Jev is slow, unavailable or unsure, a deterministic **fallback** takes over (the trained DistilBERT classifier from Day 3, or a fixed tier) — and the fallback never silently promotes a request to the paid tier. The decision, its inputs and its latency go into the trace, so a later report can explain every routed request.

D · Day 2 evaluation pipeline



Every report is regenerated from stored records. The test split is not read by the runner, the scorers' tuning, or training.

Figure 15. Diagram D. Versioned datasets are loaded, deduplicated and grouped, frozen into a manifest, and split 60/20/20 by group. Only train and validation reach the runner. Scored results become immutable records from which every report is regenerated. The test split stays sealed until Day 6.

Data moves in one direction. Loaders turn each dataset row into a prompt record with a stable ID, a pinned dataset revision and the reference answer. Deduplication and grouping keep paraphrases and closely related variants together, so that a group can be assigned to exactly one split. The frozen manifest — between 2,000 and 3,000 prompts — is identified by a content hash, so any later report can say exactly which prompts it covers.

The runner sends train and validation prompts to all three tiers within explicit request, token and dollar limits, and checkpoints as it goes. Task-specific scorers turn raw outputs into outcomes. Each outcome is appended to an immutable record file and never edited. Reports, oracle labels, cost tables and audit samples are all derived from those records, which means they can be regenerated — and checked — at any time.

Architecture and milestones: consistency check

Each architectural component should appear in exactly one checkpoint where it is introduced, and no Day 1 or Day 2 checkpoint should depend on a component that arrives later. This table was checked against `docs/COMMIT_PLAN.md` and the Day 1 commits.

Component in diagrams A–D	Introduced at	Depends on · notes
Gateway API, health, model registry	D1-05 (4d6ba55)	Config (D1-04). `/readyz` reports process readiness only.
Reusable async transport	D1-06 (e4406c4)	Used by every tier from D1-10 on.

Component in diagrams A–D	Introduced at	Depends on · notes
Ollama small / medium adapters	D1-07, D1-08	Live answers pending: Ollama was not installed at preflight.
Frontier tier with spending controls	D1-09 (1ade058)	Disabled until key, model, prices and caps exist.
Explicit-tier dispatch; streaming	D1-10, D1-11	Diagram B; `model=auto` waits for the D3-17 router dispatch.
Request records	D1-13 (2d01a58)	Extended with decision fields in Day 3.
Colab vLLM experiment path	D1-15 (19bd683)	Not executed; GPU and wheel compatibility unknown.
Versioned datasets, manifest, splits	D2-01 to D2-06	No dependency on routers or Jev.
Scorers and code sandbox	D2-05, D2-07 to D2-09	Sandbox uses Docker on the Mac (already present at preflight).
Three-tier runner, oracle labels, costs	D2-11 to D2-15	Needs working backends from Day 1; frontier only with caps.
Jev decision path (diagram C)	D3-08 to D3-10, D3-17	Uses Day 2 labels for thresholds and calibration.
DistilBERT fallback	D3-03 to D3-06	Trained on Day 2 train-split oracle labels.
Embedding cache, verifier	D4-01 to D4-17	Outside this notebook's detailed scope.

One gap is worth recording: Day 2's runner (D2-11) needs at least two live backends to produce meaningful labels, and those live checks are still pending from D1-07 and D1-08. Day 2 can implement and offline-test every component, but its real measurements (D2-18) are blocked until Ollama is installed and the models answer.

PART 5

Prerequisites and setup notes

Day 1 touches packaging, HTTP, concurrency, version control and testing all at once. This part collects the practical knowledge each checkpoint assumes, then gives a setup sequence for the Mac and for Colab and a map of the repository.

Python packages, imports, virtual environments and lockfiles

A **module** is one `.py` file; a **package** is a directory of modules that can be imported by name. The gateway's code lives in `src/jev_gateway/`, so `from jev_gateway.config import from_env` works once the package is installed. The `src/` layout prevents a common trap: without it, Python can import the code straight from the working directory and tests pass even when the package would fail to install.

A **virtual environment** (`.venv/`) is a private directory of installed packages for one project, so the gateway's FastAPI version cannot collide with another project's. **pyproject.toml** declares what the project needs: here, four core dependencies (FastAPI, httpx, Uvicorn, Pydantic) with version ranges, plus optional groups — `test`, `training` (transformers, datasets, accelerate, torch) and `embeddings` (sentence-transformers, faiss-cpu) — and a console script that makes `jev` run `jev_gateway.cli:main`.

A **lockfile** (`uv.lock`) records the exact version and hash of every package, including indirect ones, that satisfied those ranges when it was created. `uv sync --locked` installs exactly that set and fails if `pyproject.toml` and the lock disagree, which is what makes "it worked on my machine" reproducible on a clean checkout.

> **TECHNICAL NOTE · WHY THE LOCK CONTAINS CUDA PACKAGES**
>
> The lockfile resolves *all* optional groups, so it lists torch and NVIDIA wheels for the `training` extra. They are not installed by a plain `uv sync`; D1-16 records that the core environment contains no torch, CUDA, NVIDIA or vLLM packages `RECORDED`. vLLM is deliberately not in `pyproject.toml` at all — it is pinned only in `requirements/colab-serving.txt` for the notebook.

HTTP requests, JSON, URLs, ports and authentication

The gateway, Ollama, vLLM, TypeSafe and the frontier provider all communicate over **HTTP**: a client sends a *request* with a method (`GET`, `POST`), a path, headers and an optional body; the server returns a *status code*, headers and a body. Bodies here are **JSON**. Status codes carry meaning the gateway relies on: 2xx success, 4xx the client's fault (422 invalid input, 429 over limit), 5xx the server's (502 bad upstream, 503 unavailable, 504 upstream timeout).

A **URL** such as `http://127.0.0.1:11434/v1/chat/completions` combines a scheme (`http`), a host (`127.0.0.1`), a **port** (11434 — which program on that host), and a path. Each server listens on its own port: gateway 8080, Ollama 11434, notebook vLLM 8000. `config.py` rejects base URLs that are not absolute HTTP(S) or that embed credentials such as `http://user:pass@host`.

Authentication for the paid APIs uses a bearer token: the header `Authorization: Bearer <key>`. The gateway reads the frontier key only from the environment variable `JEV_FRONTIER_API_KEY`, sends it only to that backend, and never logs it. Local Ollama ignores keys.

Why “localhost” differs across a Mac, a container and Colab

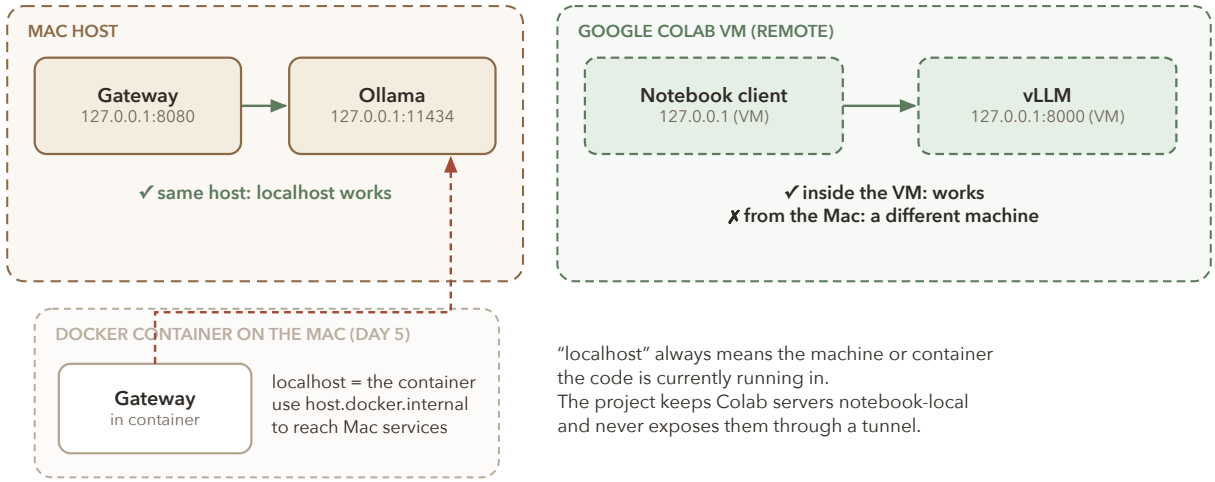


Figure 16. `localhost` (or `127.0.0.1`) always refers to the machine or container where the code is running. It works between the gateway and Ollama on the Mac, fails from inside a container unless the host address is used, and never reaches the Mac from Colab.

The address `127.0.0.1` is the *loopback* interface: traffic to it never leaves the current network namespace. On the Mac that means the gateway can reach Ollama at `127.0.0.1:11434`. Inside a Docker container on the same Mac (Day 5), `127.0.0.1` is the container itself; Docker Desktop documents `host.docker.internal` as the name for reaching services on the host. In Colab, `127.0.0.1:8000` is the vLLM server inside the Colab virtual machine — a different computer in Google's data centre. That is why the notebook runs its server, gateway code and client all in the same VM, and why the project rejects public tunnels as a way to bridge the two.

Async I/O and reusable HTTP clients

Most of a gateway's time is spent *waiting* — for a model to finish, for bytes to arrive. In synchronous code, a waiting request blocks its thread. In **asynchronous** code (`async def`, `await`), waiting hands control back to an event loop, which runs other requests until the awaited data arrives. One process can therefore hold many slow upstream calls open at once. Async does not make a model generate faster; it stops the gateway from being the bottleneck while models work.

Opening a new TCP (and, for HTTPS, TLS) connection for every request is slow. A **reusable client** — one `httpx.AsyncClient` created at startup — keeps a pool of open connections and reuses them. `BackendTransport` bounds that pool (20 connections, 10 kept alive) so a burst of requests cannot open unlimited sockets, and closes it when the app shuts down via FastAPI's lifespan hook.

Timeouts, cancellation and streaming

A **timeout** bounds how long one operation may wait. The gateway applies `JEV_TIMEOUT_SECONDS` (default 60, allowed range up to 300) to backend calls and maps an expiry to HTTP 504. A **deadline** is the related idea for a whole request: once routing and verification add steps (Days 3–4), each step must use what is left of one shared budget rather than its own fresh timeout.

Cancellation is what happens when the caller goes away. In Python's asyncio, the waiting task receives a `CancelledError`; correct code lets it propagate and releases resources in `finally` blocks — closing the upstream response so the backend connection is freed. The streaming generator in `api.py` does exactly this.

Streaming sends the answer as it is produced. OpenAI-compatible servers use server-sent events: lines of the form `data: {json}` separated by blank lines, ending with `data: [DONE]`. The gateway forwards these bytes without assembling the whole answer first, so the client sees the first words as soon as the backend produces them.

Git: working tree, staging, commits and remotes

The **working tree** is the files as they are on disk now. The **staging area** (index) holds the exact changes chosen for the next commit — `git add -p` lets you stage some hunks of a file and not others. A **commit** is a permanent snapshot with a message, author and timestamp; its hash (such as `4d6ba55`) identifies it forever. A **remote** is another copy of the repository, such as one on GitHub; nothing leaves the Mac until you `git push`.

The project's rule is one coherent checkpoint per commit, with its tests and docs, labelled by checkpoint ID (`D1-05 feat: add gateway API and health endpoints`). Two details from this repository: the Git root is the parent folder `Research Project 2`, so paths in commits begin with `llm_gateway_full_project/`; and the Colab notebook is fed with `git archive` of a committed revision, which is why uncommitted edits never reach a Colab run.

Unit tests, mocks, contract checks, real integrations and benchmarks

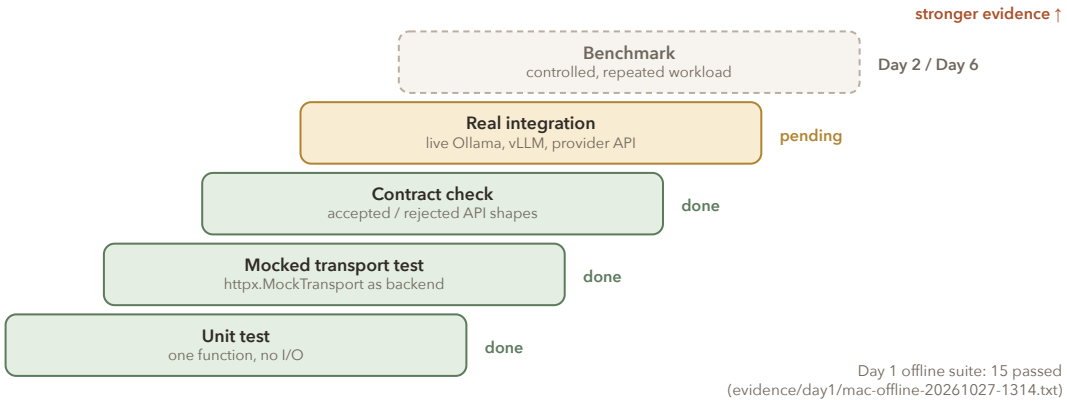


Figure 17. Kinds of evidence, from weakest to strongest claim about the real system. Day 1 has offline evidence for the bottom three rungs; real integration and benchmarks remain to be done.

- **Unit test.** One function, controlled inputs, no network — for example, an invalid URL raises `ConfigError` (`test_config.py`).
- **Mock.** A stand-in for a dependency. `httpx.MockTransport` lets tests play the role of Ollama or a provider and return chosen responses, including 500 errors, timeouts and fragmented SSE.
- **Contract check.** Verifies the public interface: which requests the API accepts, which it rejects and with what code (`test_api_contract.py`).
- **Real integration check.** The real gateway talking to a real Ollama or API. Only this proves that model names, URLs and response shapes match reality; a mock cannot. `scripts/real_check.sh` is the opt-in entry

point.

- **Benchmark.** A controlled, repeated measurement under a defined workload. It answers "how fast" or "how accurate", which none of the above can.

Setup sequence on the Mac

The commands below are grouped by purpose. Tool checks and installs come from official instructions; commands that run the project's own code are marked as such and exist in the repository as of D1-18.

PREFLIGHT · READ-ONLY CHECKS

```
sw_vers                                  # macOS version (Ollama requires macOS 14 Sonoma or
newer)
system_profiler SPHardwareDataType | grep -E "Chip|Memory"
df -h .                                  # free disk before any model download
git --version; python3 --version; docker --version; kubectl version --client
uv --version; ollama --version           # both absent at the 27 Oct preflight
```

INSTALL TOOLS · OFFICIAL INSTRUCTIONS

```
curl -LsSf https://astral.sh/uv/install.sh | sh   # or: brew install uv   (docs.astral.sh/uv)
# Ollama: download the macOS app from ollama.com, drag to Applications, start it once;
# it offers to link the `ollama` CLI into /usr/local/bin   (docs.ollama.com/macos)
```

PROJECT ENVIRONMENT · REPOSITORY COMMANDS

```
cd "llm_gateway_full_project"
uv sync --locked                         # exact core dependencies from uv.lock
uv run jev --help
uv run --extra test pytest -q            # offline suite (15 tests at D1-18)
```

MODELS AND LIVE CHECKS · REPOSITORY COMMANDS, PENDING FIRST RUN

```
ollama pull qwen2.5:1.5b                 # ~986 MB download
ollama list; ollama ps
uv run jev doctor                        # reports installed/loaded models, frontier status
uv run jev serve                         # terminal 1: gateway on 127.0.0.1:8080
uv run jev smoke --tier small            # terminal 2: one bounded request
ollama pull qwen2.5:7b                   # only after checking free disk (~4.7 GB)
uv run jev smoke --tier medium; ollama ps # then record residency and memory pressure
```

Configuration is read from environment variables; `config/mac.env.example` lists them with safe defaults. Secrets go in an untracked `.env` (ignored by Git) and are exported in the shell. The frontier tier stays disabled until `JEV_FRONTIER_MODEL`, `JEV_FRONTIER_API_KEY`, both per-million-token prices and positive request, token and USD caps are all set.

Setup sequence for Colab

1. On the Mac, commit the work, then build a source archive from that commit and record its digest. The repository's notebook README gives:

   ```
   git archive --format=zip -o jev-source.zip HEAD llm_gateway_full_project
   shasum -a 256 jev-source.zip
   ```

2. In Colab, choose a GPU runtime explicitly and run `notebooks/colab_preflight.py` or the notebook's first cell to record Python, platform, `nvidia-smi` GPU name, memory and driver.
3. Paste the SHA-256 into the notebook, upload the zip, and let the notebook verify the hash and extract safely.
4. Install the package from the extracted source plus the pinned vLLM. Check the pin against the GPU and CUDA reported in step 2 and vLLM's installation page first: the repository pins `vllm==0.18.0`, while PyPI listed 0.32.0 as the latest release on 28 Oct 2026. If installation fails, record the facts and update the pin in a new checkpoint.
5. Run the serve, readiness, request and export cells; download the JSON result; run the cleanup cell. Results are labelled Colab and kept separate from Mac results.

Proposed and current repository layout

Path	State	Responsibility
`src/jev_gateway/`	exists	The installable application package: API, configuration, transport, budget, tracing, diagnostics, Ollama discovery, CLI.
`tests/`	exists	Offline tests (15 at D1-18). Real checks are opt-in and live in `scripts/`.
`config/`	exists	Example environment profiles for Mac and Colab. No secrets.
`requirements/`	exists	Environment-specific pins that must not enter the core install (vLLM for Colab).
`notebooks/`	exists	Colab preflight script and the serving notebook; README with the archive procedure.
`scripts/`	exists	Opt-in real-path checks (`real_check.sh`).
`docs/`	exists	Plan, commit plan, status, decisions, environments, experiments, runtime profiles – the engineering evidence ledger. This notebook lives in `docs/engineering-notebook/`.
`evidence/day1/`	exists	Raw, timestamped outputs that status claims cite.
`prompts/`	exists	Per-day build instructions from the starter.
`src/jev_gateway/eval/`	PLANNED	Day 2 record schema, dataset loaders, manifest and split builder, scorers, runner, reports.
`data/manifests/`	PLANNED	Frozen manifest files and their hashes (prompt IDs and provenance, not bulk dataset copies).
`results/`	PLANNED	Run outputs (already in `.gitignore`); summaries copied into `evidence/` when cited.
`sandbox/`	PLANNED	Container definition for isolated code scoring.

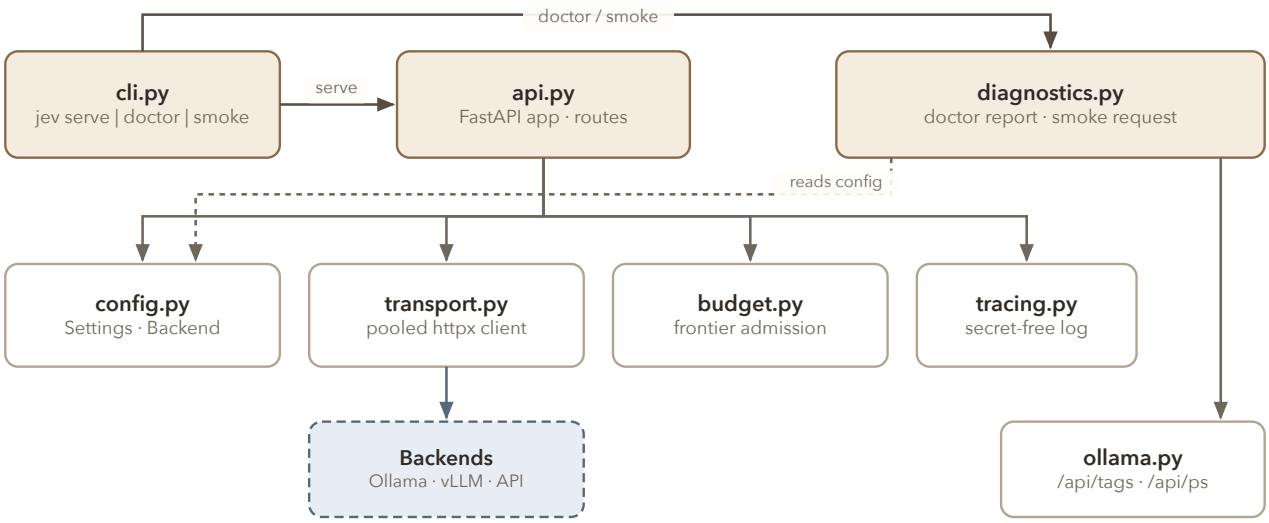


Figure 18. Module map of `src/jev_gateway` at D1-18. Arrows point from a module to what it uses. `api.py` is the hub for serving; `diagnostics.py` and `ollama.py` support the `doctor` and `smoke` commands.

PART 6

Day 1 — the serving foundation

Day 1 builds a gateway that can serve all three tiers by explicit name, stream, fail clearly, stay within a budget, and leave a record of every request. It deliberately stops short of automatic routing.

Overview and recorded state

All eighteen Day 1 checkpoints were committed on 27 October 2026 between 13:03 and 13:11 (America/Toronto), as `7764899` through `4e6f7e0` `RECORDED`. The final offline run, after `uv sync --locked`, reported **15 passed, 1 warning, 0.16 s**; the warning is a third-party Starlette deprecation notice about the test client. The raw output is `evidence/day1/mac-offline-20261027-1314.txt`.

Everything that needs a live model is still open. Ollama was not installed at the preflight, so `jev doctor` reported `ConnectError` for both local tiers; the frontier tier reports `configured: false`; and no Colab cell has run. In the status figure below, amber marks checkpoints whose code is committed but whose completion evidence requires a live check.



Figure 19. The eighteen Day 1 checkpoints in six blocks of three, coloured by recorded status in `docs/STATUS.md` at commit `4e6f7e0`.

Each checkpoint below follows the same structure: what to understand first, what the change introduces, why it belongs at that point in the build, what evidence establishes completion, one specific common mistake, and what the repository records.

BLOCK 1 Scope, environments and the package

D1-01 Define full scope and runtime profiles `7764899`

UNDERSTAND	The difference between a scope map (every deliverable and where it runs) and a schedule. Which work belongs to the Mac, to Colab, and to external APIs.
CHANGE	A full-scope contract in `AGENTS.md` and `docs/RUNTIME_PROFILES.md`, which maps each original feature to a checkpoint range and environment and states that a running notebook is never a permanent backend.
WHY HERE	Every later checkpoint decides where something runs. Settling that first prevents features from being dropped quietly when they become inconvenient.
EVIDENCE	A table in which every original deliverable has a checkpoint range and an environment, with no "optional" or "stretch" labels.
COMMON MISTAKE	Writing "the medium tier runs in Colab" into the application's design. That turns an experiment environment into a production dependency and leads straight to the notebook-dependent architecture rejected in Part 1.
RECORDED	"Full scope mapped." Six feature groups mapped to D1–D6 ranges.

D1-02 Record Mac and Colab environment preflights `8f69a1c`

UNDERSTAND	Which machine facts constrain the design: unified memory, free disk, OS version, toolchain. Why a remote environment needs its own report from its own host. What to omit (serial numbers, UUIDs, usernames).
CHANGE	A dated, sanitised Mac report in `docs/ENVIRONMENT.md`, listing the commands used, and `notebooks/colab_preflight.py`, which prints Python, platform, `nvidia-smi` GPU name/memory/driver and `nvcc` as JSON.
WHY HERE	Model selection (D1-07, D1-08) depends on memory and disk; the vLLM pin (D1-15) depends on the GPU and CUDA driver.
EVIDENCE	A Mac report with values and sources; a Colab report produced on an allocated Colab GPU and labelled separately.
COMMON MISTAKE	Writing a GPU type such as "T4" into the plan before a session is allocated. Colab's available GPUs vary over time.
RECORDED	MacBook Pro (Mac15,6), M3 Pro with 11 CPU cores, 18 GB unified memory, macOS 26.5.2, 18 GiB free of 460 GiB. Python 3.14.5, Docker CLI and `kubectl` present; `uv`, `ollama`, `kind` absent. Colab preflight not yet run.

D1-03 Initialize the reproducible package and CLI `2448290`

UNDERSTAND	`pyproject.toml`, the `src/` layout, console scripts, optional extras, and what a lockfile guarantees.
CHANGE	Package `jev-gateway` 0.1.0 built with hatchling, requiring Python ≥3.11 and <3.15; four core dependencies; `test`, `training` and `embeddings` extras; a `jev` command with `serve`, `doctor` and `smoke` subcommands; `uv.lock`.
WHY HERE	Every later module and test imports the package. A lock from the first commit means any later failure can be reproduced exactly.
EVIDENCE	In a fresh environment: `uv sync --locked`, `import jev_gateway`, and `jev --help` all succeed.
COMMON MISTAKE	Adding `torch` or `transformers` to the core dependencies "for later". The Mac gateway would then download gigabytes of ML libraries it never uses.

RECORDED	Fresh virtual environment install, import, CLI help, and uv lock.

BLOCK 2 Contracts: configuration, API and transport

D1-04 Validate backend configuration and resource limits `10dbdbc`

UNDERSTAND	Fail-fast validation; immutable settings objects; holding secrets *by reference* (the name of an environment variable) rather than by value; limits as policy.
CHANGE	`config.py`. `Backend(tier, model, base_url, api_key_env)` requires an absolute HTTP(S) URL without embedded credentials and a non-empty model. `Settings` requires a timeout in (0, 300] seconds, `max_output_tokens` in 1–8192, and non-negative frontier caps. `frontier_enabled` is true only when a model, a key and positive caps and prices all exist. `from_env()` parses typed values and reports which variable was malformed.
WHY HERE	The API and transport both rely on settings being valid. A mistyped URL caught at startup is far cheaper than a 502 discovered later.
EVIDENCE	Tests for a `file://` URL, a non-integer limit, and a frontier configuration that has a key but no caps (must stay disabled).
COMMON MISTAKE	Defaults that enable the paid tier, or keeping the key itself in a settings field that appears in logs and error messages. Here the key is read from the environment only when a request is built.
RECORDED	Config invalid and disabled cases pass.

D1-05 Add the API contract, health, readiness and model registry `4d6ba55`

UNDERSTAND	Liveness ("the process is up") versus readiness ("it can serve"); schema validation; why unknown fields are rejected rather than ignored.
CHANGE	Pydantic models with `extra="forbid"`: roles `system`, `developer`, `user`, `assistant`; text content of at least one character; `model` limited to `small`, `medium`, `frontier`; `max_tokens` 1–8192; `temperature` 0–2. Endpoints `/healthz`, `/readyz` and `/v1/models` (each tier with its backend model and enabled flag).
WHY HERE	The contract is what clients code against and what tests check. Fixing it before the transport avoids reshaping both later.
EVIDENCE	Contract tests: health is OK; three models listed; `model="auto"` → 422; image content → 422; frontier while unconfigured → 503.
COMMON MISTAKE	A readiness endpoint that reports backends healthy without checking them. Here `/readyz` says `"backend_availability": "unchecked"`, and live availability is left to `jev doctor`.
RECORDED	Contract tests pass. A brief real loopback server returned HTTP 200 from `/healthz`; binding a socket needed elevated sandbox permission (the default sandbox denied it with EPERM).

D1-06 Implement reusable asynchronous backend transport `e4406c4`

UNDERSTAND	Connection pooling and limits; client lifecycle; building a request once and sending it; injecting a fake transport for tests.
CHANGE	`BackendTransport` wraps one `httpx.AsyncClient` with a timeout and limits (20 connections, 10 keep-alive). `complete()` posts to `{base_url}/chat/completions` and raises on non-2xx; `open_stream()` returns an open streaming response; the bearer header is added only when a key exists. The app closes the client on shutdown.
WHY HERE	All three tiers share one HTTP behaviour. Building it once, before any adapter, keeps timeouts and limits consistent.

EVIDENCE	A `MockTransport` test checks the path `/v1/chat/completions`, the forwarded body, and a clean close.
COMMON MISTAKE	Creating a new `AsyncClient` per request. It looks harmless in tests, but under load it discards pooling and can leak sockets.
RECORDED	Mock HTTP forwarding passes.

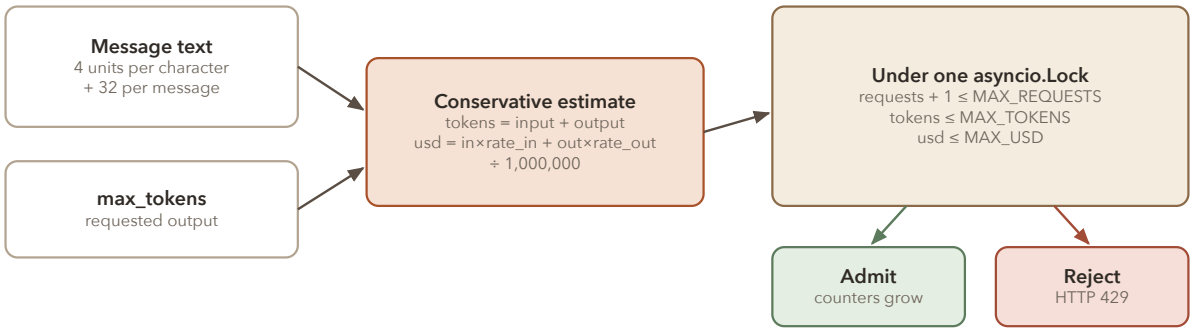
BLOCK 3 The three generation tiers

D1-07 Connect the small Ollama model `ef3b3bc`

UNDERSTAND	Ollama's two surfaces: the native API (`/api/tags` for installed models, `/api/ps` for loaded ones) and the OpenAI-compatible `/v1/chat/completions`. Model tags and why the name must match exactly.
CHANGE	`ollama.py`: `model_status()` reads `/api/tags` and reports whether the configured tag is installed, with runtime label `native-ollama-mac`. `docs/MAC_MODELS.md` gives install, pull and check steps. Generation itself goes through the shared transport.
WHY HERE	The smallest model is the cheapest way to prove the whole path end to end before adding memory-heavy or paid tiers.
EVIDENCE	One bounded real answer through the gateway, with the model ID and runtime label recorded.
COMMON MISTAKE	Treating "pulled" as "working". A completed download says nothing about whether the model loads and answers within the timeout.
RECORDED	Adapter offline-tested; real answer pending because Ollama was absent at preflight.

D1-08 Connect the medium model and inspect memory and residency `94d6717`

UNDERSTAND	Residency, `keep_alive`, `size_vram`, memory pressure, and cold versus warm requests (Part 3).
CHANGE	`loaded_models()` reads `/api/ps` and returns each loaded model's name, `size_vram` and `expires_at`; `jev doctor` includes it. Docs give the observation procedure: one bounded medium request, then `ollama ps`, Activity Monitor memory pressure, cold and warm load times, and whether both tags stay loaded.
WHY HERE	It tests hypothesis H4. Day 2's evaluation plan depends on whether both tiers can stay loaded or must run one at a time.
EVIDENCE	A real medium answer with explicit residency and switching observations, or a documented hardware blocker.
COMMON MISTAKE	Reporting only warm latency as "the medium model's latency", or assuming both models are co-resident because both were downloaded.
RECORDED	Residency adapter offline-tested; live memory and load measurements pending.



Reservations are never released, so counts only rise within a process. The guard is per-process and approximate; it complements, and does not replace, spending limits in the provider's account.

Figure 20. Frontier admission in `budget.py` (D1-09). The estimate is deliberately high; the check-and-reserve happens under one lock so that concurrent requests cannot overshoot the caps between them.

D1-09 Connect the frontier API with spending controls `1ade058`

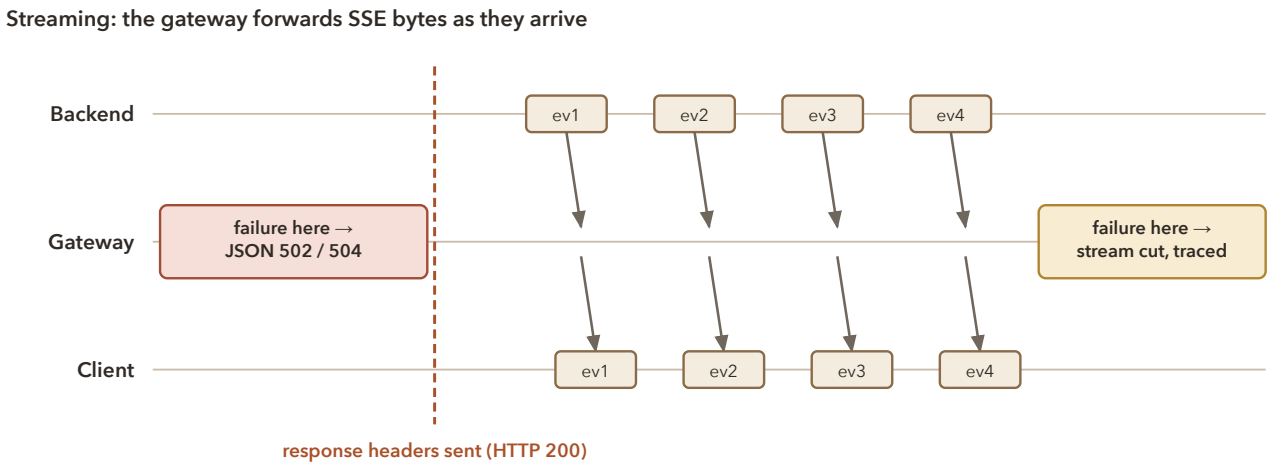
UNDERSTAND	Per-token pricing; admission control (deciding before the call) versus accounting (measuring after); process-local versus account-level limits; why counters need a lock under async concurrency.
CHANGE	`FrontierBudget.reserve()` estimates input as four units per character plus 32 per message, adds `max_tokens`, prices both with the configured per-million rates, then — under an `asyncio.Lock` — admits only if requests, tokens and dollars all stay within caps. Over the cap: HTTP 429. Not configured: 503.
WHY HERE	The paid path must be safe before D1-10 wires dispatch and makes it reachable.
EVIDENCE	Offline: with a cap of one request, a second reservation is refused. Real: a single request only once a key, model, current prices and positive caps are configured.
COMMON MISTAKE	Recording spend after the response returns. Twenty concurrent requests would all pass the check before any was counted.
RECORDED	Admission tested; paid calls disabled. Reservations are never released, so counters only rise within a process, and the estimate is not provider billing (noted in `docs/DECISIONS.md`).

BLOCK 4 The request path: responses, streaming, failures

D1-10 Implement explicit-tier nonstreaming responses `27a16e0`

UNDERSTAND	Translating a client request into a backend payload; why the backend body is preserved; carrying metadata in headers; the output-token policy.
CHANGE	The chat handler resolves the tier, builds `{model: backend.model, messages, max_tokens, stream}` (with `temperature` only if given), defaults `max_tokens` to the server policy and rejects larger values with 422, sends via the transport, and returns the backend JSON with `x-request-id`, `x-jev-tier` and `x-jev-backend-model`.
WHY HERE	It is the first complete path. Streaming and error handling both build on it.
EVIDENCE	A `medium` request reaches the backend with `"model":"qwen2.5:7b"`, the answer arrives unchanged, and the tier header is set.
COMMON MISTAKE	Forwarding the tier name (`"medium"`) as the backend's model name. Ollama would answer 404 because no model has that name.

RECORDED	Explicit dispatch mock-tested; live backend pending.



After the first byte the status code is already 200, so a mid-stream failure can only end the stream and be recorded as stream_interrupted.

Figure 21. The streaming failure boundary (D1-11, D1-12). Before the first byte reaches the client, an upstream error can still become a clean 502 or 504. After headers are sent, the status is fixed at 200, so a failure can only cut the stream; the trace records it as `stream_interrupted`.

D1-11 Implement streaming response forwarding `a95c350`

UNDERSTAND	SSE framing; network chunks that split or merge events arbitrarily; HTTP headers committing at the first byte.
CHANGE	Streaming requests open the upstream with `open_stream()` and return a `StreamingResponse` that yields the upstream's raw bytes as they arrive, with `text/event-stream` and `cache-control: no-cache`. The generator finishes the trace on completion or interruption and always closes the upstream.
WHY HERE	It reuses the D1-10 path and fixes the boundary that D1-12's error policy must respect.
EVIDENCE	A test sends three events across arbitrary chunk boundaries and checks that both content pieces and the final `data: [DONE]` arrive intact, with the tier header present.
COMMON MISTAKE	Parsing and re-serialising each event in the gateway. It adds latency and breaks when a network chunk ends mid-event; forwarding bytes avoids both.
RECORDED	Fragmented SSE and completion tested; live streaming pending.

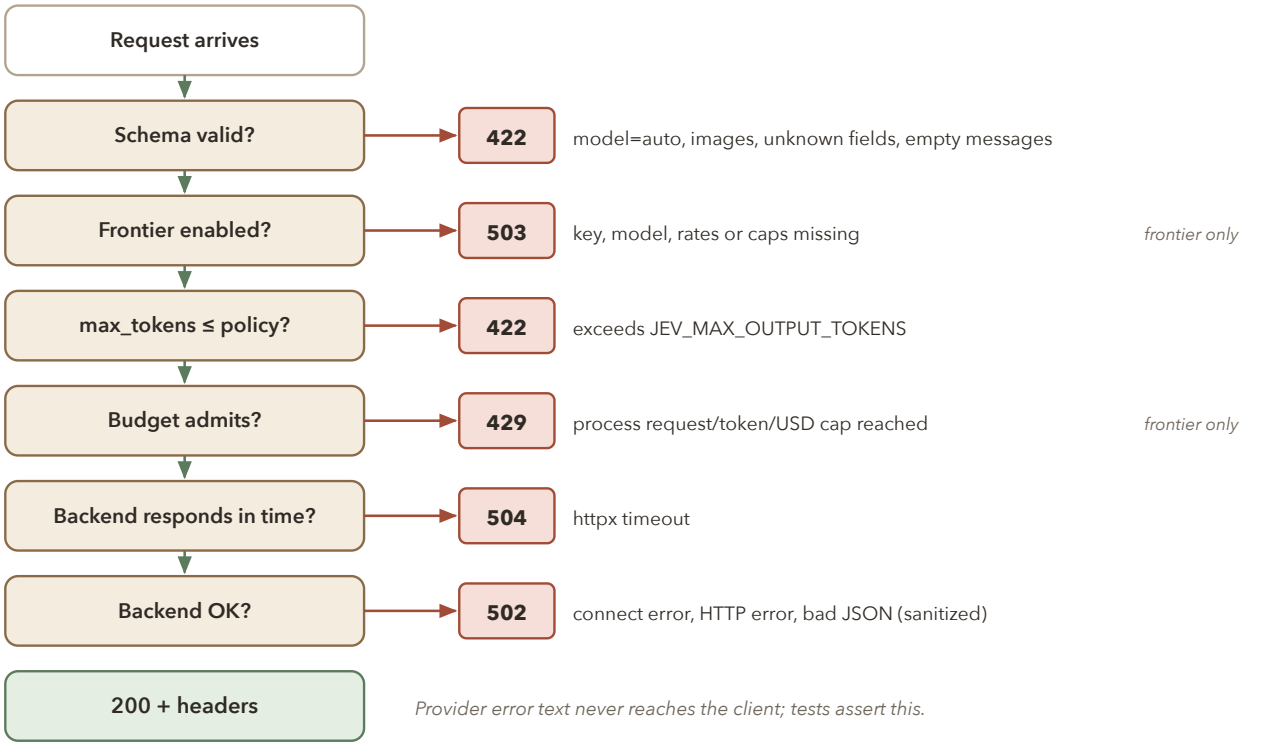


Figure 22. How the Day 1 gateway maps each failure to a status code, in the order the checks run. Provider error text never reaches the client.

D1-12 Handle deadlines, cancellation and backend errors `6d438ec`

UNDERSTAND Mapping failure types to status codes; why error bodies are sanitised; what is possible before versus after headers; letting cancellation propagate.

CHANGE `httpx.TimeoutException` → 504 "backend deadline exceeded"; HTTP status errors, connection errors and invalid JSON → 502 "backend request failed". For streams, `open_stream()` checks status before any byte is forwarded, so an upstream 500 still becomes a clean 502. Any exception during the stream, including cancellation, is traced as `stream_interrupted`, re-raised, and the upstream is closed.

WHY HERE Only once both paths exist can failure behaviour be specified for both.

EVIDENCE Tests: an upstream 500 containing "secret provider detail" becomes a 502 without that text, streaming and non-streaming; a read timeout becomes 504.

COMMON MISTAKE Catching `CancelledError` with a broad `except` and carrying on. The backend keeps generating for a client that has already left.

RECORDED Timeout and sanitised-error tests pass. No test yet simulates a client disconnecting mid-stream; cancellation handling rests on code structure (`except BaseException … raise`, `finally: aclose()`).

BLOCK 5 Evidence: traces, diagnostics and the notebook

D1-13 Record request traces, usage and timings `2d01a58`

UNDERSTAND Request IDs; monotonic versus wall-clock time; allowlisting fields rather than blocklisting; one structured line per event.

CHANGE	`RequestTrace` assigns a `uuid4` hex ID and a UTC start time, measures `duration_ms` with a monotonic clock, and logs one JSON line with tier, backend model, status, usage restricted to `prompt_tokens`, `completion_tokens` and `total_tokens`, and `cost_status` (`reserved` for frontier, `unpriced` for local). The ID is returned in `x-request-id`.
WHY HERE	Diagnostics, the notebook and Day 2 all need comparable records, so the record format comes first.
EVIDENCE	A test passes a usage dict containing an extra `secret` key and checks that it never appears in the log, and that the logged ID matches.
COMMON MISTAKE	Timing with `time.time()`. The wall clock can jump (NTP adjustment), producing negative or inflated durations; `time.monotonic()` cannot.
RECORDED	Secret-free trace structure tested. The record currently holds one total duration; per-stage timestamps (for example, first upstream byte), named in the commit plan, are not yet captured.

D1-14 Add doctor and bounded smoke commands `cb90aa4`

UNDERSTAND	Diagnostics should touch real systems, be bounded in time and cost, and explain failure in words — never silently substitute test doubles.
CHANGE	`jev doctor` reports whether the frontier tier is configured and, for each local tier, queries `/api/tags` and `/api/ps` with a 3-second timeout, reporting the reason on failure; it exits 0 only if both local models are installed. `jev smoke --tier T` sends one request with `max_tokens: 16` and a 15-second timeout, then prints the tier, request ID, backend model and answer.
WHY HERE	These commands collect the live evidence that D1-07, D1-08 and the handoff require.
EVIDENCE	With Ollama absent, `doctor` explains the failure; `smoke` against a closed port prints "smoke failed" and exits 1.
COMMON MISTAKE	A smoke command that defaults to the frontier tier or sends a long prompt. A routine check then becomes a paid call.
RECORDED	Doctor reports both models unavailable (`ConnectError`) and `frontier.configured=false`; raw JSON in `evidence/day1/mac-doctor-20261027-1314.json`.

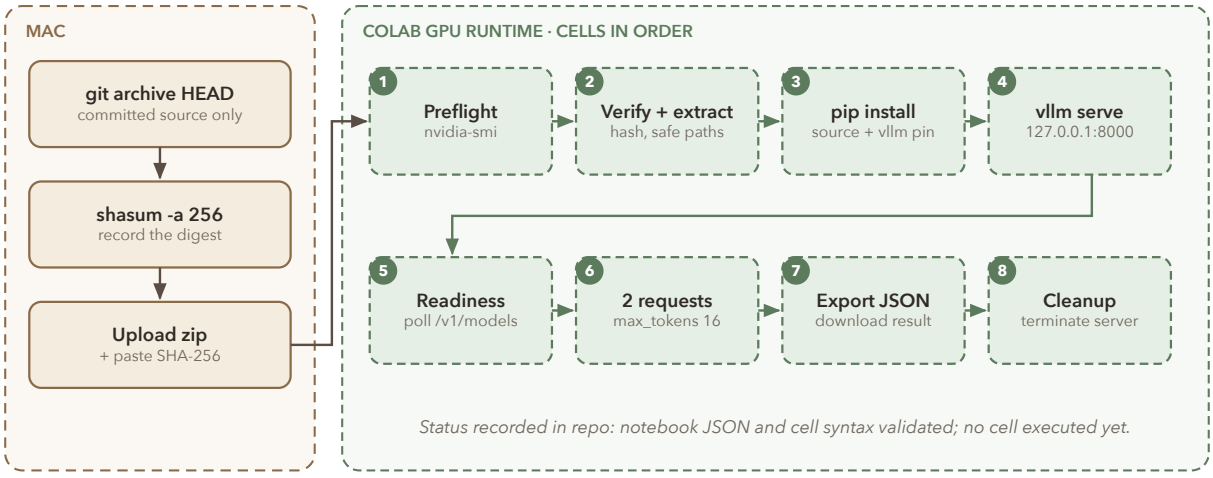


Figure 23. The Colab serving notebook (D1-15). Source enters only as a hashed archive of a committed revision; the server binds to loopback inside the VM; results are exported before the session ends.

D1-15 Add a source-importing Colab serving notebook `19bd683`

UNDERSTAND	Why notebooks drift from repositories; pinning source by content hash; key `vllm serve` flags; why export comes before cleanup.
CHANGE	`day1_vllm.ipynb`: GPU preflight; upload and SHA-256 check of `jev-source.zip` with safe extraction (no absolute or `..` paths); `pip install` of the source plus `vllm==0.18.0`; `vllm serve Qwen/Qwen2.5-1.5B-Instruct` on `127.0.0.1:8000` with `--max-model-len 2048` and `--gpu-memory-utilization 0.75`; readiness polling for up to 240 s; two prompts at temperature 0 with `max_tokens` 16; JSON download; termination.
WHY HERE	The gateway contract and the profile split now exist, so the notebook can install and import the same code rather than copy it.
EVIDENCE	The notebook structure validates, plus a record of which cells actually ran, on which GPU, with which archive hash.
COMMON MISTAKE	Pasting gateway code into notebook cells "to save time". From then on there are two implementations, and results from one say nothing about the other.
RECORDED	Notebook JSON and cell syntax validate; no Colab cell executed. The vLLM pin is a starting point and predates the current release.

BLOCK 6 Portability, verification and handoff

D1-16 Separate Mac and Colab profiles and dependencies `59a5eac`

UNDERSTAND	Configuration profiles versus code branches; keeping GPU-only packages out of the core install; ignoring local secrets.
CHANGE	`config/mac.env.example` (Ollama on 11434, Qwen tags, timeout, caps commented out) and `config/colab.env.example` (loopback 8000/8001, Hugging Face model IDs); `requirements/colab-serving.txt` for vLLM; lock updated for the extras; `.env` added to `.gitignore`.
WHY HERE	The notebook (D1-15) made the second environment concrete; the profiles make the difference explicit before integration tests target both.
EVIDENCE	A core environment on the Mac contains no torch, CUDA, NVIDIA or vLLM packages.
COMMON MISTAKE	One shared `requirements.txt` for Mac and Colab. Either the Mac install fails on CUDA wheels, or the notebook's pins leak into the app.
RECORDED	Core venv has no torch/CUDA/NVIDIA/vLLM packages.

D1-17 Verify offline contracts and available real integrations `3e6320f`

UNDERSTAND	What an offline integration test can and cannot show; why real checks are opt-in; reporting the scope of each piece of evidence.
CHANGE	`test_profile_integration.py` dispatches through the full app with the small tier pointed at the Mac URL and then the Colab URL, checking URL, model and request ID. `scripts/real_check.sh` runs `doctor`, then `smoke` for small and medium against live services.
WHY HERE	All components exist; this is the first check that they work together under both profiles.
EVIDENCE	The offline suite passes, and whatever real-path evidence is available is recorded with its exact scope.
COMMON MISTAKE	Making real checks part of the default `pytest` run. They become flaky, slow, and potentially paid — and people learn to ignore failures.

RECORDED	15 passed, 1 warning, 0.16 s (`mac-day1-offline-20261027-1314`). Real models pending.

D1-18 Write setup documentation and the evidence handoff `4e6f7e0`

UNDERSTAND	Who reads a handoff: someone who must reproduce the result without asking. What a status ledger is for.
CHANGE	README quickstart and API description; `docs/STATUS.md` with a commit and verification note per checkpoint and a list of unmet gates; `docs/DECISIONS.md`; run IDs in `docs/EXPERIMENTS.md`; raw files in `evidence/day1/`.
WHY HERE	It closes the day: the next session, or another person, must know exactly what is proven and what is pending.
EVIDENCE	A clean reader can find the commands, the required model downloads, the evidence files and the next action.
COMMON MISTAKE	Marking a checkpoint "done" when its only evidence is a mocked test. Here the ledger says "mock-tested; live pending".
RECORDED	Quickstart and evidence ledger prepared; clean-reader walkthrough pending. Next action recorded: install Ollama, pull `qwen2.5:1.5b`, run `doctor`, `serve`, `smoke --tier small`.

DAY 1 GATES STILL OPEN

(1) Install and start native Ollama, check disk, pull the small then medium model, run bounded answers, and record model IDs, load times, `ollama ps` and memory pressure. (2) Run the Colab notebook on an allocated GPU and export its JSON. (3) Configure a frontier key, model, current prices and positive caps before any paid call. (4) Do a clean-reader install and real-path smoke test.

Journal · Day 1: a gateway before a router

Conceptual entry, ready to copy. It explains the Day 1 system as designed and committed; it reports no model output, because none has been observed yet.

The first day's goal was not routing. It was a dependable place to put routing later: a single service that speaks the same chat API as the models behind it, knows exactly which backends exist, and leaves a record of everything it does. The client still chooses the tier by name — small, medium or frontier — and anything else, including a request for automatic routing, is refused at the door. That restriction is deliberate. A router cannot be judged until every tier can be reached reliably and every outcome can be measured.

The gateway has four parts. Configuration describes each tier as a URL and a model name, checks them at startup, and keeps the paid tier switched off until a key, a model, current prices and positive spending caps are all present. The API layer defines what a valid request looks like and rejects everything else with a clear status code. A single asynchronous HTTP client, created once and shared, forwards requests and keeps a bounded pool of connections open, so the gateway can wait on many slow models at once without opening a new connection for each. A trace object gives every request an ID and records its tier, backend model, status, token usage and duration.

The most useful idea of the day was the boundary that streaming creates. Before the first byte goes back to the client, a failing backend can still be turned into an honest error: 504 if it ran out of time, 502 if it failed. After the first byte, the HTTP status is already 200, and the only honest thing left is to stop the stream and record that it was interrupted. That is why the gateway checks the backend's status before forwarding anything, forwards raw bytes rather than re-parsing events, and closes the upstream connection in a finally block whether the stream ended normally, failed, or was cancelled because the client left.

The paid tier needed a different kind of care. A budget that is counted after each response returns can be overrun by concurrent requests that all pass the check before any is recorded. The gateway therefore reserves an overestimate of tokens and dollars before calling the provider, under a lock, and refuses the request if the reservation would cross a cap. It is a guard against runaway loops inside one process, not a bill, and it complements rather than replaces limits set in the provider's account.

Two environments are kept apart on purpose. The Mac runs the application and, through Ollama, the local models. Colab is a laboratory: it receives a hashed archive of a committed revision, installs that exact code, serves a model on its own loopback address, sends two small requests, exports the result and shuts the server down. The Mac never sends traffic to Colab, so a notebook session ending can never break the application, and every number carries the name of the environment that produced it.

The evidence so far is honest but limited. The offline suite passes, and it covers configuration errors, the request contract, forwarding, fragmented streams, sanitised errors, timeouts, budget refusal and secret-free traces. Each of those tests replaces the model server with a controlled fake,

which proves the gateway's logic and nothing about the real servers. The diagnostic command reached for Ollama and found nothing listening, because Ollama was not yet installed. No model has answered through the gateway.

The next technical question is physical rather than logical: on an 18 GB Mac, can the 1.5B and 7B models both stay loaded, and if not, what does switching between them cost? The answer decides how Day 2's evaluation is scheduled, and it is the first measurement that will be taken from the real machine.

PART 7

Day 2 — evaluation and baselines

Day 2 builds the instrument that every later claim depends on: a frozen set of prompts with known answers, scorers that can be trusted, a runner that collects outcomes from all three tiers within budget, and reports that can be regenerated from raw records. All of it is PLANNED; none has been implemented in the repository yet.

The ideas Day 2 depends on

Why the three task families differ

The benchmark mixes three public datasets because each stresses something different, and routing is only interesting if the tiers' strengths differ by task.

Dataset	What a prompt looks like	How it is scored	Licence · revision at access
GSM8K	A grade-school word problem needing several arithmetic steps.	Extract the final number and compare with the reference after the dataset's `####` marker.	MIT · `openai/gsm8k` @ `740312a`
MMLU	A four-option multiple-choice question from one of 57 subjects.	Extract one letter A–D and compare with the reference index.	MIT · `cais/mmlu` @ `c30699e`
MBPP	A short Python task with a description and assert-based tests.	Run the generated function against the tests in a sandbox.	CC BY 4.0 · `google-research-datasets/mbpp` @ `4bb6404`

Maths rewards multi-step reasoning and punishes a single slip; multiple-choice rewards stored knowledge and has a 25% chance floor; code requires exact syntax and correct behaviour. A small model might do respectably on multiple-choice and poorly on maths — hypothesis H1 — and that per-task difference is what a router can exploit. Revisions above are the Hugging Face commit hashes at access on 28 October 2026; D2-02 to D2-04 pin them in the loaders.

Training, validation and test

The manifest is split three ways. The **training** split (60%) is what the DistilBERT router learns from in Day 3. The **validation** split (20%) is used for every choice that involves looking at results: prompt templates, Jev question wording, thresholds, scorer fixes. The **test** split (20%) is used once, on Day 6, after every one of those choices is frozen. If the test set influences any decision, its numbers stop being an estimate of performance on unseen prompts and become a description of how well the choices fit those particular prompts.

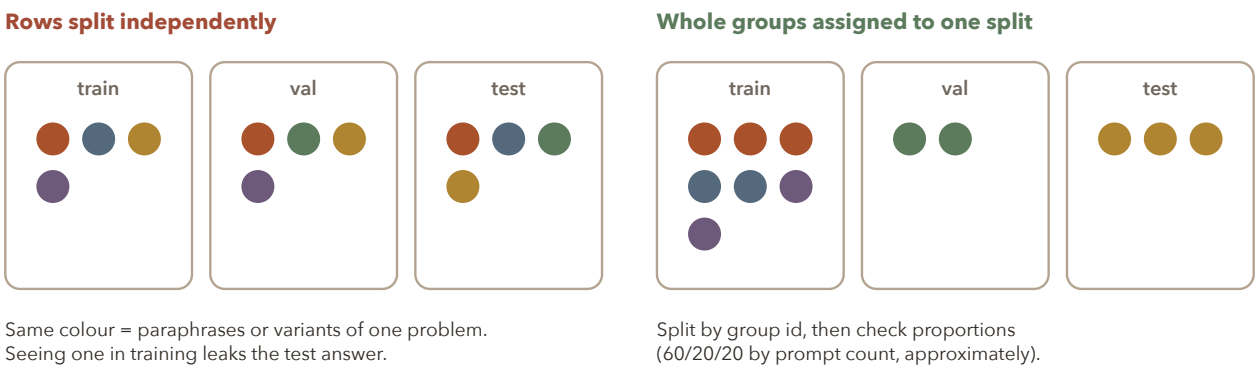


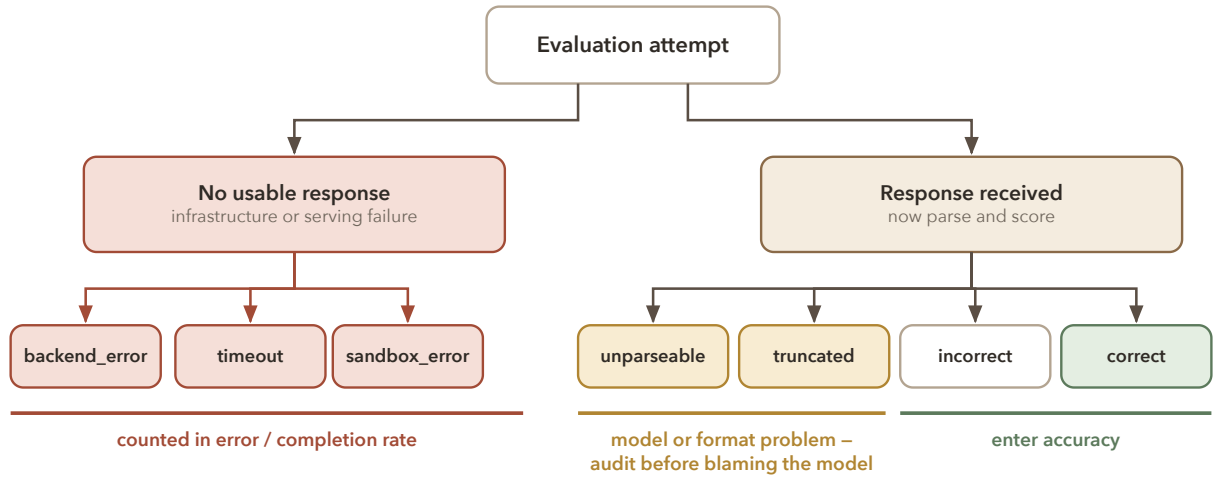
Figure 24. Splitting rows independently can put variants of one problem on both sides of the split. Assigning whole groups prevents this kind of leakage.

Data leakage and related-example grouping

Leakage is any path by which information about the evaluation data reaches the thing being evaluated. Here the most likely path is related examples: two GSM8K problems that differ only in numbers, MMLU questions that share a passage, MBPP tasks that are near-duplicates. If one is in training and its twin is in test, the router can look clever by remembering. The defence is to group related prompts first (exact duplicates by normalised-text hash, near-duplicates by a similarity check, shared-source items by their source ID) and then assign *groups* to splits. Proportions become approximately rather than exactly 60/20/20, which is the right trade.

A second form of leakage cannot be removed: these are public benchmarks, and any of the generation models may have seen them during pre-training. That inflates absolute accuracy for all tiers. Comparisons between tiers and routers on the same prompts remain meaningful; claims about absolute capability should be cautious.

Scoring errors versus model errors



Each outcome is stored as its own value. Accuracy, completion rate, and error rate state their denominators.

Figure 25. Outcome taxonomy. Infrastructure failures are not wrong answers; unparseable and truncated responses need auditing before they are blamed on the model.

When a tier's accuracy looks surprisingly low, there are two very different explanations. A **model error** means the model's answer was wrong. A **scoring error** means the model's answer was right but the scorer failed to recognise it — for example, the maths extractor took the last number in "The answer is 42. Check: 6 × 7 = 42." correctly, but in "So 42 apples, which is 3 dozen and 6" it would take 6. Scoring errors are systematic: they distort every comparison built on top of them. That is why each outcome type is stored separately, and why D2-10 exports samples for a human to check before any thresholds are chosen.

Why generated code needs isolation

Scoring MBPP means running code a model wrote. That code can loop forever, allocate all memory, delete files, open network connections, or spawn processes. Most of the time it will do none of those things, but the evaluation runs thousands of programs, and one bad one must not be able to damage the Mac or hang the run. The plan is a disposable container for each check with no network, memory, CPU and process limits, a non-root user, a temporary working directory and a hard wall-clock timeout. A timeout becomes a recorded `timeout` outcome, not a stuck evaluation.

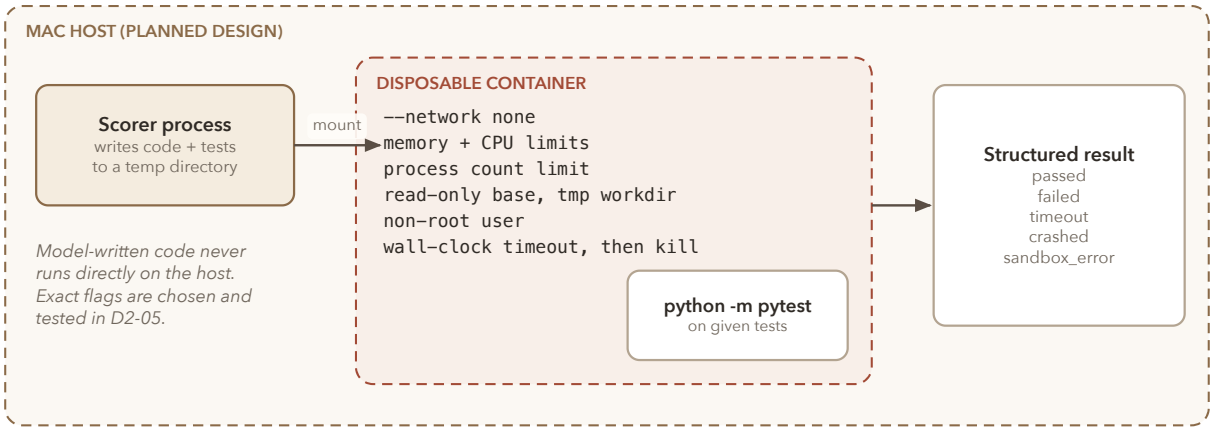


Figure 26. Planned isolation for code scoring (D2-05). Exact Docker flags are a D2-05 decision and are verified there with deliberately hostile fixtures.

What the hindsight oracle can and cannot establish

prompt	small	medium	frontier	oracle label
gsm8k-0412	✓	✓	✓	small
mmlu-1185	✗	✓	✓	medium
mbpp-0077	✗	✗	✓	frontier
gsm8k-0903	✗	✗	✗	none
mmlu-0261	✓	✗	✓	small

Illustrative rows. The label is the cheapest tier that was observed to succeed, so it uses outcomes that a live router cannot see before generating. Note the last row: correctness is not monotonic in size.

Figure 27. Deriving oracle labels from observed outcomes. Rows and results are illustrative.

After every tier has answered every train and validation prompt, each prompt can be labelled with the **cheapest tier that was observed to answer correctly**, or `none` if no tier did. Routing every prompt to its label gives the **hindsight oracle**: the best cost and accuracy any router could have achieved on these outcomes.

The oracle *can* establish headroom: if it costs much less than always-frontier at similar accuracy, routing has room to help. Its labels *can* train the DistilBERT router. It *cannot* be deployed, because it uses outcomes nobody knows before generating. It is tied to one sample per prompt — with temperature above zero a tier might succeed on a rerun — and to the scorer's judgement. And it says nothing about latency. It is a bound on the measured data, reported as such.

Why sequential answer collection is not a live routing latency benchmark

On a memory-limited Mac the sensible way to collect outcomes is one tier at a time: load the small model, run every prompt, then the medium model, then the frontier API. That gives clean quality labels and clean per-tier generation timings. It does not measure what a routed request would experience, because it never alternates models (so never pays switching cost), never contends for memory with the other tier, never includes a Jev or DistilBERT decision on the path, and runs with whatever batching the runner chose rather than real arrival patterns. Live routing latency is a separate measurement (Day 3's overhead benchmark and Day 5's load tests). Day 2 reports quality runs and serving baselines as different things, with different metadata.

API usage costs versus provisioned GPU capacity costs

An API bill is proportional to tokens: no requests, no cost. A provisioned GPU is billed by time: the same hourly rate whether it serves one request or ten thousand. Mixing the two produces nonsense, so they are reported separately.

```
Illustrative prices and workloads — not quotes and not project measurements
API:  input $1.00 / M tokens, output $4.00 / M tokens; request = 400 in, 300 out
      cost = 400 × 1.00/1e6 + 300 × 4.00/1e6 = $0.0004 + $0.0012 = $0.0016 per request
GPU:  $1.20 per hour, whatever the load
      at 2 requests/s sustained: 7,200 req/h → $1.20 / 7,200 ≈ $0.00017 per request
      at 0.05 requests/s:          180 req/h → $1.20 / 180   ≈ $0.0067 per request
      idle:                          0 req/h → $1.20 per hour for nothing
Routing 30% of traffic away from that GPU lowers its utilisation, not its bill.
```

Jev adds a third cost shape: TypeSafe charges per input token only, at $0.042 per million, so a decision's cost is set by state size and question count DOCUMENTED. For the Mac tiers, the honest entry is "unpriced": electricity and hardware wear exist but are not measured. D2-14 keeps measured spend, modelled capacity cost and unknown rates in separate columns.

The eighteen Day 2 checkpoints



Figure 28. Day 2 checkpoints in six blocks of three. None has started in the repository.

BLOCK 1 Records and the first two datasets

D2-01 Define immutable evaluation records `planned`

UNDERSTAND	Schemas as contracts; append-only data; why an error is a value and not a missing row.
PLANNED CHANGE	A frozen record type with prompt ID, task, split, manifest hash, tier, backend model and revision, generation settings, template version, raw output, finish reason, extracted answer, outcome (one of the taxonomy values), usage, timings and cost fields, run ID and timestamp. Records are written as JSON lines and never edited.
WHY HERE	Loaders, scorers, the runner and reports all produce or consume this record; defining it first prevents four incompatible formats.
EVIDENCE	Fixtures for each outcome validate; a backend error and a wrong answer remain distinguishable after a round trip through the file.
COMMON MISTAKE	Dropping failed calls from the results. Accuracy then silently excludes the hardest or slowest prompts and looks better than it is.

D2-02 Load versioned GSM8K prompts `planned`

UNDERSTAND	Dataset revisions; deterministic IDs; where the reference answer lives (`#### <number>` at the end of the solution).
PLANNED CHANGE	A loader that pins the dataset revision, assigns stable IDs (for example `gsm8k-test-0412`), stores the question and the parsed reference number, and records licence and source.
WHY HERE	Maths has the simplest reference answers, so it is the easiest place to prove the loader-to-record path.
EVIDENCE	A known sample loads with the same ID, text and answer on two machines; provenance is recorded.

COMMON MISTAKE	Loading the dataset's moving default branch. A later upstream fix changes prompts under the same IDs.

D2-03 Load versioned MMLU prompts `planned`

UNDERSTAND	Subject subsets; consistent prompt templates; how an answer index maps to a letter.
PLANNED CHANGE	A loader for selected subjects that renders each question with options labelled A–D in the dataset's order, stores the correct letter, and records subject, revision and template version.
WHY HERE	It exercises templating and answer mapping, which the multiple-choice scorer (D2-08) depends on.
EVIDENCE	Tests confirm option order and the letter derived from the reference index on known items.
COMMON MISTAKE	Shuffling options for "fairness" without storing the permutation. The reference letter no longer matches the rendered prompt.

BLOCK 2 Code tasks, isolation and the frozen manifest

D2-04 Load MBPP tasks and test specifications `planned`

UNDERSTAND	What the model must produce (a function with a specific name and signature) and what the tests assert.
PLANNED CHANGE	A loader that stores the task text, the test list, any setup code, the expected function name, the task ID, and licence metadata.
WHY HERE	The sandbox (D2-05) needs real task specifications to be tested against.
EVIDENCE	Dataset and licence metadata and task IDs recorded; a reference solution passes its own tests in the sandbox.
COMMON MISTAKE	Leaving the expected function name out of the prompt. Correct code under a different name then fails every test.

D2-05 Isolate generated-code execution `planned`

UNDERSTAND	Containers as process and filesystem isolation; network namespaces; resource limits; the difference between a test failure and an infrastructure failure.
PLANNED CHANGE	A runner that writes code and tests to a temporary directory and executes them in a disposable, network-disabled, resource-limited container as a non-root user, returning `passed`, `failed`, `timeout`, `crashed` or `sandbox_error`.
WHY HERE	No model-written code may run before isolation exists, and the code scorer (D2-09) builds on it.
EVIDENCE	Hostile fixtures — an infinite loop, a memory bomb, a network call, a file write outside the work directory — are each contained and classified correctly.
COMMON MISTAKE	"Just for a quick check", running generated code with `exec()` in the evaluator's own process.

D2-06 Freeze a deduplicated manifest and grouped 60/20/20 splits `planned`

UNDERSTAND	Duplicates and near-duplicates; group IDs; seeded assignment; why "frozen" means hashed.
PLANNED CHANGE	Select 2,000–3,000 prompts across the three tasks, group related items, assign groups to train/validation/test with a fixed seed, and write a manifest of IDs, groups, splits and provenance with its content hash. Reading the test split requires an explicit flag.

WHY HERE	All three loaders exist, and the split must be fixed before any result is seen.
EVIDENCE	No group spans two splits; the manifest hash is reproducible from the same inputs and seed; the runner refuses test prompts without the flag.
COMMON MISTAKE	Regenerating the split after looking at early results. Every later number then depends on that choice.

BLOCK 3 Scoring

D2-07 Implement math answer extraction and scoring

planned

UNDERSTAND	Normalising numbers (commas, currency symbols, trailing units, decimals and fractions); explicit parser outcomes.
PLANNED CHANGE	An extractor that prefers an explicit final-answer pattern requested by the prompt template, falls back to clearly defined rules, and returns either a number or `unparseable`; then exact numeric comparison.
WHY HERE	It is the simplest scorer and the first that real outputs will hit.
EVIDENCE	Edge-case tests ("$1,250.00", "3/4", "about 12", several numbers in a row) pass; the audit in D2-10 confirms behaviour on real outputs.
COMMON MISTAKE	Always taking the last number in the text. Answers that end with a check or a unit conversion are then scored wrong.

D2-08 Implement multiple-choice scoring

planned

UNDERSTAND	Accepted answer forms; ambiguity; why guessing in the scorer is worse than recording "unparseable".
PLANNED CHANGE	A parser that accepts documented forms ("B", "(B)", "Answer: B", the option text matching exactly one choice) and returns `unparseable` for answers naming several letters or none.
WHY HERE	It shares the outcome taxonomy with D2-07 and must be audited with it.
EVIDENCE	Malformed, truncated and incorrect answers remain distinguishable in tests.
COMMON MISTAKE	Matching the first capital A–D anywhere in the text. "A good answer is C" scores as A.

D2-09 Integrate isolated code-task scoring

planned

UNDERSTAND	Extracting code from a chat answer (fenced blocks, prose around code); mapping sandbox results to outcomes.
PLANNED CHANGE	Connect completions to the D2-05 sandbox: extract the code block, combine it with the task's tests, run it, and store a structured result including the sandbox outcome and test counts.
WHY HERE	It combines D2-04 and D2-05, and completes the set of scorers.
EVIDENCE	Known pass, fail, timeout and syntax-error fixtures are classified correctly.
COMMON MISTAKE	Counting a sandbox infrastructure failure, such as Docker not running, as the model's code failing.

D2-10 Export samples for manual scoring audits

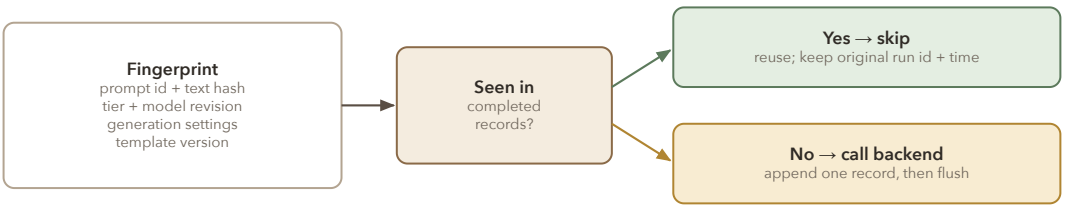
planned

UNDERSTAND	Why human review catches systematic scorer bugs that unit tests miss; stratified sampling.
PLANNED CHANGE	Export 30–50 development-split answers per task, stratified by outcome and tier, in a format a person can read and mark; record corrections and the scorer changes they cause.
WHY HERE	Before any threshold or router is tuned on scored outcomes.
EVIDENCE	A documented audit with disagreement counts and the resulting scorer fixes.
COMMON MISTAKE	Auditing only answers marked wrong. False positives — wrong answers scored correct — are invisible that way.

D2-11 Implement bounded evaluation across all three tiers

planned

UNDERSTAND	Concurrency limits; retries for transient errors only; per-run request, token and dollar caps; deterministic generation settings.
PLANNED CHANGE	A runner that sends selected splits through the gateway to each configured tier, with fixed settings (for example temperature 0), bounded concurrency and retries, and caps. Paid tiers are skipped unless a positive run budget is configured.
WHY HERE	Records, loaders and scorers all exist; this connects them to the Day 1 gateway.
EVIDENCE	A small development run completes for each available tier; no paid job starts without an explicit budget.
COMMON MISTAKE	Retrying wrong answers or timeouts until they succeed. That changes what is being measured.



An interrupted run leaves complete lines plus at most one partial line. On resume, the partial line is discarded, completed fingerprints are skipped, and only missing work is sent. Changing any fingerprint field (e.g. a new model revision) makes a new experiment, not a resume.

Figure 29. Planned resume logic (D2-12). A completed result is reused only when its full fingerprint matches; anything else is a new experiment.

D2-12 Add checkpointing and safe resumption

planned

UNDERSTAND	Fingerprints; atomic appends and flushing; partial lines after a crash.
PLANNED CHANGE	Fingerprint each unit of work (prompt ID and text hash, tier, model revision, generation settings, template version); on restart, discard a trailing partial line, skip completed fingerprints, and continue.
WHY HERE	Long runs over thousands of prompts on a laptop or in Colab will be interrupted.
EVIDENCE	A deliberately interrupted development run resumes without duplicate or corrupted records, and reused results keep their original run ID and time.

COMMON MISTAKE	Keying completion on prompt ID alone. After a model update, old answers are silently reused as if new.

BLOCK 5 Oracle, cost and quality reports

D2-13 Derive observed hindsight-oracle labels

planned

UNDERSTAND	Tier order by cost; what to do when no tier is correct; why the oracle is a bound, not a policy.
PLANNED CHANGE	For each train and validation prompt, compute the cheapest tier observed to succeed, or `none`; store the label with the record set it came from; report the oracle's accuracy and cost.
WHY HERE	It needs complete per-tier outcomes, and Day 3 training needs its labels.
EVIDENCE	Labels regenerate from records; the report states that the oracle is an offline bound for these measured outcomes.
COMMON MISTAKE	Mapping `none` to `frontier`. The classifier is then trained to send unanswerable prompts to the most expensive tier.

D2-14 Calculate complete inference costs

planned

UNDERSTAND	Token-billed versus time-billed resources; cost attribution per request; unknown rates.
PLANNED CHANGE	A cost model with separate fields for frontier usage (from returned usage and configured prices), decision and verification calls (from Day 3 on), and provisioned-capacity scenarios with an explicit hourly rate and utilisation. Local Mac tiers stay "unpriced" unless a rate is supplied.
WHY HERE	Reports in D2-15 and every Day 3 comparison need cost on a common basis.
EVIDENCE	Unknown rates remain unknown in the output; idle capacity appears as a cost, not zero.
COMMON MISTAKE	Treating self-hosted tiers as free. That makes "route everything local" look optimal regardless of quality.

D2-15 Generate per-tier quality and completion reports

planned

UNDERSTAND	Accuracy versus completion rate versus error rate, each with its denominator; per-task breakdowns; sample counts.
PLANNED CHANGE	A report generator that reads records and outputs, per tier and per task, accuracy, completion and error rates, outcome counts, sample sizes and costs, stamped with the manifest hash and run IDs.
WHY HERE	It closes the quality half of the day; its output is the baseline every router must beat.
EVIDENCE	Deleting the report and regenerating it from records gives an identical file.
COMMON MISTAKE	Reporting accuracy over completed responses only, without the completion rate beside it.

BLOCK 6 Serving baselines and real measurements

D2-16 Define controlled serving benchmark workloads

planned

UNDERSTAND	Closed-loop versus open-loop load; concurrency versus arrival rate; warm-up; prompt and output length control.

PLANNED CHANGE	A workload runner, separate from the quality runner, with configurable prompt-length buckets, fixed output lengths, concurrency or arrival rate, warm-up requests, and recorded model load state and cache state.
WHY HERE	Quality numbers are now in hand; performance needs its own controlled experiment rather than timings borrowed from evaluation runs.
EVIDENCE	Workload definitions are stored with every result, including client scheduling, warm-up and output controls.
COMMON MISTAKE	Reusing evaluation timings as a latency benchmark. Output lengths vary per prompt, so differences in tokens generated swamp differences in serving speed.

D2-17 Report latency, throughput, errors and sample counts

planned

UNDERSTAND	The definitions in Part 8: time to first output, end-to-end latency, event gaps versus token timing, percentiles, throughput units.
PLANNED CHANGE	Metric computation with explicit denominators: per-request time to first output and end-to-end latency (p50 and p95, stating the percentile method), output tokens per second from usage counts, request throughput over a stated window, error rate, and sample counts.
WHY HERE	It turns D2-16's raw timings into the baselines Day 3 and Day 6 compare against.
EVIDENCE	Each metric's formula and denominator is printed in the report; failed requests appear alongside successful-request latency.
COMMON MISTAKE	Computing latency percentiles over successful requests only. A configuration that fails its slow requests then looks faster.

D2-18 Preserve actual train/validation measurements; keep the test held out

planned

UNDERSTAND	What can be completed within hardware and budget limits; reporting progress honestly; protecting the test split.
PLANNED CHANGE	Run the real evaluation on train and validation for every available tier within caps, preserve raw records and the manifest hash, and report coverage towards the 2,000+ prompt target per tier.
WHY HERE	It is the day's deliverable: real outcomes that Day 3 can train and tune on.
EVIDENCE	Raw records, run IDs and coverage counts per tier in `docs/EXPERIMENTS.md`; test-split access count of zero.
COMMON MISTAKE	"Just a quick look" at test-split accuracy. Once seen, it cannot be unseen, and every later decision is suspect.

DEPENDENCY CARRIED OVER FROM DAY 1

D2-01 to D2-17 can be implemented and tested offline with fixtures. D2-18 needs live tiers: at least the Ollama small and medium models answering through the gateway (D1-07, D1-08), and for the frontier tier a configured key and run budget. Until then, coverage is reported as zero rather than estimated.

Journal · Day 2: building an instrument worth trusting

Conceptual entry, ready to copy. Day 2 has not been implemented; this entry explains the design and its reasons and reports no results.

Day 1 built a way to ask any of three models a question. Day 2 builds a way to know whether their answers were right, and that turns out to be the harder problem. Every later claim in the project — that one router saves money, that Jev's probabilities mean what they say, that a cache does not return wrong answers — rests on correctness labels produced here. If those labels are biased, every comparison built on them inherits the bias, however careful the later work is.

The benchmark draws on three public datasets because the tiers should differ by task. GSM8K word problems need several correct arithmetic steps and a single final number. MMLU questions offer four options across many subjects and reward stored knowledge. MBPP tasks ask for a short Python function that must pass hidden tests. Each dataset is pinned to a specific revision and each prompt gets a permanent ID, so the same prompt means the same thing in every run and on every machine.

Before any model sees a prompt, the prompts are grouped and split. Related items, such as paraphrases or problems that differ only in their numbers, are placed in one group, and whole groups go to training, validation or test. Training will teach the fallback router; validation will absorb every decision that involves looking at results; the test split stays sealed until the final day. Splitting by group rather than by row costs a little precision in the 60/20/20 proportions and buys protection against the most likely form of leakage.

Scoring gets as much attention as generation. A maths answer has to be found in a paragraph of reasoning, a letter has to be found without guessing, and generated code has to run somewhere it can do no harm — in a disposable container without network access, with limits on memory, processes and time. Each attempt ends in one of several distinct outcomes: correct, incorrect, unparseable, truncated, timed out, or failed for infrastructure reasons. Keeping these apart is what lets a low score be traced to its cause. A human then reads a sample of scored answers from each task, because a scorer bug is systematic and no amount of model comparison will reveal it.

Once each tier has answered the training and validation prompts, each prompt receives an oracle label: the cheapest tier observed to get it right, or none. The oracle is the best routing that was possible on these particular outcomes. It shows how much room there is for a real router, and its labels become training data, but it is not a router, because it uses answers no one has at request time.

Cost and speed are measured with the same care. A paid API charges per token; a rented GPU charges per hour whether busy or idle; the Mac's own tiers are honestly marked unpriced. These are kept in separate columns. Collecting answers one tier at a time is the right way to get quality labels on a memory-limited laptop, but it is not a measurement of how fast a routed request would be, because it never switches models or makes a routing call. Serving performance therefore gets its own controlled workloads, with defined prompt and output lengths, warm-up, and percentiles reported alongside error rates and sample counts.

The work divides cleanly into what can be built and tested now and what must wait. Records, loaders, splits, scorers, the sandbox, the runner, resumption and reports can all be written and checked against fixtures. Real outcomes need real models answering through the gateway, which brings the project back to the first open question from Day 1: getting Ollama running on the Mac and seeing the small model answer.

PART 8

Essential measurement notes

A short reference for the quantities Days 2, 3 and 6 will report. Every numerical example in this part is ILLUSTRATIVE; none is a project measurement.

Latency: end to end, first output, and what happens between

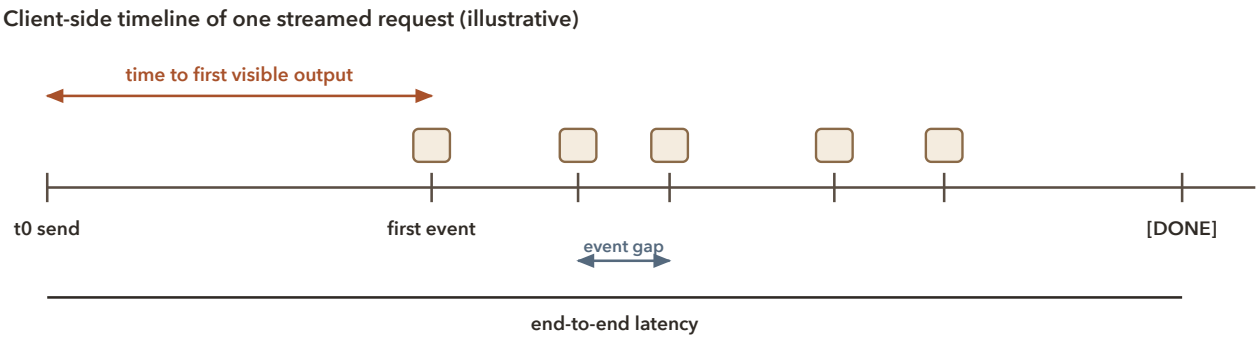


Figure 30. Client-side timings for one streamed request. Time to first visible output and end-to-end latency are measured from when the client sends; event gaps are measured between consecutive content events.

End-to-end latency is the time from the client sending a request to receiving the last byte (for streams, the `[DONE]` event). It is what a non-streaming user experiences and what a deadline bounds. State where it is measured: the client's view includes the network and the gateway; the gateway's trace `duration_ms` starts after the request has been received and parsed.

Time to first visible output is the time until the first event that contains answer text. It is close to, but not the same as, the server's time to first token: some servers send an initial event that carries only the role or metadata, and the network and gateway add their own delay. Name which one is reported.

Stream-event gaps versus token timings. An SSE event can carry one token, several, or none. The gap between events is therefore a delivery timing, not a per-token generation timing, unless the interface guarantees one token per event. Per-token rates should come from token counts in `usage`, not from counting events.

Average time per output token describes the decode phase for one request:

```
TPOT = (end-to-end − time to first output) ÷ (output tokens − 1)
Illustrative: end-to-end 3.40 s, first output at 0.40 s, 121 output tokens (from usage)
  TPOT = (3.40 − 0.40) ÷ 120 = 0.025 s = 25 ms per token  →  about 40 tokens/s for this
request
```

p50 and p95

A latency distribution is usually skewed: most requests are similar, a few are much slower. The **p50** (median) is the value half the requests are at or below; the **p95** is the value 95% are at or below. The mean is a poor summary because one stalled request can move it a lot. Percentiles from small samples are fragile: with 20 requests, p95 is determined by roughly one request. Always report the sample count and the method (nearest-rank, or linear interpolation).

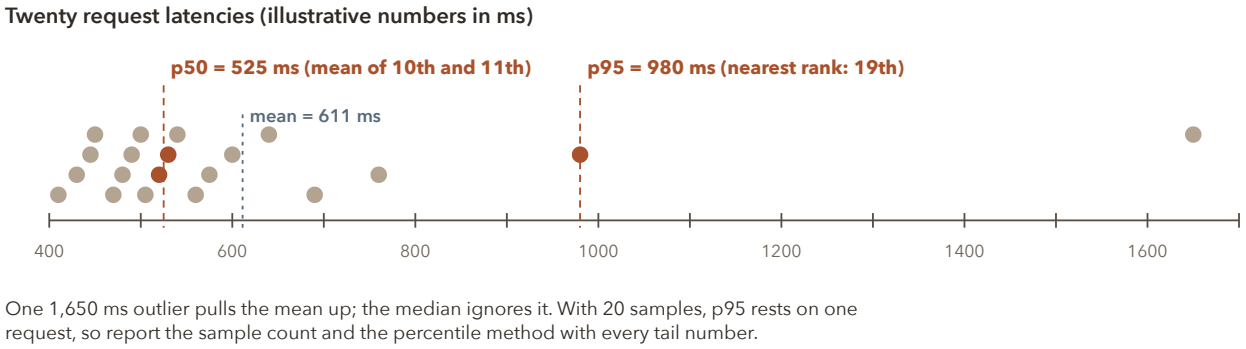


Figure 31. Median, p95 and mean for twenty illustrative latencies. The median uses the average of the 10th and 11th values; p95 uses the nearest-rank method.

Request and output-token throughput

```
Illustrative 10-minute window (600 s), fixed concurrency of 4
  completed requests 540, failed requests 18
  request throughput      = 540 ÷ 600 s       = 0.90 requests/s   (state: completed only)
  output tokens (usage)   = 81,000
  output-token throughput = 81,000 ÷ 600 s    = 135 tokens/s
  error rate              = 18 ÷ (540 + 18)   = 3.2%
```

Throughput needs a stated window, a stated load (concurrency or arrival rate), and the error rate next to it. Higher throughput at higher concurrency usually comes with higher per-request latency; report both, never one alone.

Accuracy, completion rate and error rate

```
Illustrative: 500 prompts sent to one tier
  infrastructure errors (timeout, backend error)        40
  responses received                                   460
    correct 390 · incorrect 40 · unparseable 22 · truncated 8
  accuracy (over all attempts)    = 390 ÷ 500 = 78.0%
  completion rate                 = 460 ÷ 500 = 92.0%
  error rate                      =  40 ÷ 500 =  8.0%
  accuracy among completed        = 390 ÷ 460 = 84.8%   ← must be labelled; never reported
alone
```

The headline accuracy uses all attempted prompts as the denominator, so a tier cannot look better by failing hard prompts. Unparseable and truncated responses count as not correct in accuracy, and are also reported separately because they may point to scorer or template problems.

Actual versus modelled cost

Actual cost is money spent, computed from provider-reported usage and the prices in force at the time, ideally reconciled with the provider's billing. **Modelled cost** is a scenario: "if this workload ran on a GPU rented at r per hour at utilisation u". Both are useful; they belong in different columns with their assumptions stated. The gateway's budget reservation is neither — it is a conservative admission estimate. Unknown prices stay unknown; they are never filled with a guess.

Cold versus warm runs

Report whether each measurement started with the model unloaded (cold) or loaded (warm), how many warm-up requests were discarded, and the server's caching state. On the Mac, record `ollama ps` before and after a run. A cold measurement answers "how long after idle does the first user wait?"; a warm one answers "how fast is steady-state serving?". Both are legitimate; mixing them in one distribution is not.

Mocked versus real evidence

Evidence	What it can establish	What it cannot
Mocked test	The gateway's own logic: mapping, validation, error handling, record format.	That a real server accepts the request, returns that shape, or responds in time.
Real integration check	That configured URLs, model names and response shapes match a live service.	Speed or quality beyond the single request observed.
Replayed / offline analysis	Router comparisons on stored outcomes; oracle bounds.	Live latency, switching cost, or behaviour under load.
Controlled benchmark	Latency and throughput under the stated workload, environment and load state.	Performance under a different workload or on different hardware.

Each result in the project should say which of these it is, which environment produced it (Mac or Colab), and which commit and configuration it used.

Sources

All sources were accessed on **28 October 2026**. Official documentation was preferred; where only the repository is cited, the claim is about this project's own recorded state.

TypeSafe (Jev) — official documentation

1. Introduction. https://docs.typesafe.ai/introduction
2. Quick start. https://docs.typesafe.ai/introduction/quickstart
3. System One. https://docs.typesafe.ai/concepts/system-one
4. State. https://docs.typesafe.ai/concepts/state
5. Choice. https://docs.typesafe.ai/primitives/choice
6. Score. https://docs.typesafe.ai/primitives/score
7. Noul. https://docs.typesafe.ai/primitives/noul
8. Box (image primitive, "dreamer-ready"). https://docs.typesafe.ai/primitives/box
9. Confidence. https://docs.typesafe.ai/confidence
10. Models (versions, price, context, aliases, rate limits). https://docs.typesafe.ai/models
11. AI primer (RLCD and calibration). https://docs.typesafe.ai/introduction/machine-learning-primer
12. Request rules and errors. https://docs.typesafe.ai/api/errors
13. Python SDK: basic usage; answers and responses. https://docs.typesafe.ai/sdk/python/usage/basic · https://docs.typesafe.ai/sdk/python/api/types/responses
14. typesafe-sdk package listing (version 0.7.5 at access). https://pypi.org/project/typesafe-sdk/

Serving runtimes and tools

15. Ollama for macOS (requirements, install, file locations). https://docs.ollama.com/macos
16. Ollama OpenAI compatibility. https://docs.ollama.com/api/openai-compatibility
17. Ollama FAQ (context length, keep_alive, concurrency, ollama ps). https://docs.ollama.com/faq
18. Ollama API: list running models (/api/ps). https://docs.ollama.com/api/ps
19. Ollama library: qwen2.5 tags and sizes. https://ollama.com/library/qwen2.5
20. vLLM GPU installation. https://docs.vllm.ai/en/stable/getting_started/installation/gpu/
21. vLLM package listing (latest 0.32.0 at access). https://pypi.org/project/vllm/
22. uv installation. https://docs.astral.sh/uv/getting-started/installation/
23. Docker Desktop networking (host.docker.internal). https://docs.docker.com/desktop/features/networking/
24. Google Colab FAQ (resource limits, runtime lifetime). https://research.google.com/colaboratory/faq.html

Models and datasets

25. Qwen2.5-1.5B-Instruct model card and config (Apache-2.0). https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct
26. Qwen2.5-7B-Instruct model card and config (Apache-2.0). https://huggingface.co/Qwen/Qwen2.5-7B-Instruct
27. DistilBERT base uncased (Apache-2.0). https://huggingface.co/distilbert/distilbert-base-uncased
28. GSM8K dataset (MIT). https://huggingface.co/datasets/openai/gsm8k
29. MMLU dataset (MIT). https://huggingface.co/datasets/cais/mmlu
30. MBPP dataset (CC BY 4.0). https://huggingface.co/datasets/google-research-datasets/mbpp

Repository evidence

31. Commit plan and full scope. `docs/COMMIT_PLAN.md` · `docs/PLAN.md` · `docs/RUNTIME_PROFILES.md` · `AGENTS.md`
32. Day 1 status, decisions, environments, experiments. `docs/STATUS.md` · `docs/DECISIONS.md` · `docs/ENVIRONMENT.md` · `docs/EXPERIMENTS.md` · `docs/MAC_MODELS.md`
33. Raw Day 1 outputs. `evidence/day1/mac-offline-20261027-1314.txt` · `evidence/day1/mac-doctor-20261027-1314.json`
34. Source code and tests at commit 4e6f7e0. `src/jev_gateway/` · `tests/` · `notebooks/` · `config/`